



Aug 26, 2026

Testing Report

Key Insights From TestSprite's Autonomous AI Testing Agent

Report Contents

1	Executive Summary	03
2	Frontend UI Test Results	03

27

Test Runs

40.0%

Pass Rate

23

Issues

2h

Saved By TestSprite

Table of Contents

- 3 Executive Summary
 - 3 High-Level Overview
 - 3 Key Findings

- 3 Frontend UI Test Results
 - 3 Test Coverage Summary
 - 4 Test Execution Summary
 - 5 Test Execution Breakdown

Executive Summary

1 High-Level Overview

OVERVIEW	
Project Name	QR MENU SYSTEM
Project Type	Frontend
Total Test Cases	27
Base URL	https://qr-menu-product.onrender.com
Pass / Fail / Blocked	Frontend: 4 Passed / 6 Failed / 17 Blocked

2 Key Findings

Test Summary

The executable subset shows mixed stability: 4 tests passed, 6 failed, and 12 were blocked. Because blocked cases were caused mainly by the app itself rather than harness limitations, the customer-facing journey is currently unreliable; however, the score is computed only on the executable subset, excluding the 12 blocked tests. The strongest signal is that admin and waiter flows are partially functional, while customer access/session flows are not. Repeated failures around QR access, demo sign-in, and category creation suggest the product is close in some areas but still has important state, validation, and demo-data issues that affect real user experience.

What could be better

The biggest weakness is customer access: multiple tests hit the same persistent error page, "Erişim Hatası" / "Bu QR kod artık geçerli değil", and retries never reached the menu. That blocks ordering, cart updates, payments, and session-claim flows. Waiter authentication is also broken: the demo credential "Garson · garson@demo.com" returns "E-posta veya şifre hatalı". On the admin side, creating a category failed because "E2E Test Kategori" never appeared after clicking "Oluştur", and revenue review lacks the expected date-range selector. Several blocked cases indicate the app is missing a working demo/session path, not just flaky tests.

Recommendations

Prioritize fixing the customer demo access path first, because it is blocking most end-to-end coverage. Make `/menu/demo-business-id/1`` load a valid demo session or provide a working fallback QR/demo entry so the menu, cart, order, and payment flows can be exercised. Next, verify the waiter demo account credentials and seed data so "Garson · garson@demo.com" can sign in successfully. Then inspect the category-creation modal flow and the payments page to ensure the create action actually persists and the revenue view exposes a date-range filter. After each fix, re-run the matching tests by title to confirm the original failures are gone.

Frontend UI Test Results

1 Test Coverage Summary

This report summarizes the frontend UI testing results for the application. TestSprite's AI agent automatically generated and executed tests based on the UI structure, user interaction flows, and visual components. The tests aimed to validate core functionalities, visual correctness, and responsiveness across different states.

CATEGORY	TEST CASES	PASS/FAIL RATE
Customer Ordering	5	0 Pass / 0 Fail / 5 Blocked
Admin Operations	4	1 Pass / 1 Fail / 2 Blocked
Waiter Order Management	5	2 Pass / 2 Fail / 1 Blocked
Customer Menu & QR Access	4	0 Pass / 3 Fail / 1 Blocked
Customer Payments	4	1 Pass / 0 Fail / 3 Blocked
Session Access & Table Claim	5	0 Pass / 0 Fail / 5 Blocked

Note

The test cases were generated using real-time analysis of the application's UI hierarchy and user flows. Some visual and functional validations were adapted dynamically based on runtime DOM changes.

2 Test Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
Customer Ordering			
Adjust item quantity before ordering	Validates that a customer can update item quantities in the cart and keep the final order summary in sync.	High	BLOCKED
Submit an order with special instructions	Validates that a customer can include notes for an order and submit it through the QR menu flow.	Medium	BLOCKED
Browse menu and add items to cart	Validates that a customer can open the QR menu, choose menu items, and add them to the order cart.	High	BLOCKED
Review menu categories before ordering	Validates that a customer can switch between menu categories and still begin an order from the QR menu experience.	Low	BLOCKED
Place a complete customer order	Validates that a customer can build an order from the QR menu and submit it successfully from the order flow.	High	BLOCKED
Admin Operations			
Open admin demo dashboard	Verify an admin can enter the demo environment and reach the main admin dashboard.	High	PASSED
Review revenue summary	Verify an admin can open the revenue view and see revenue metrics for operational oversight.	High	BLOCKED
Add a new staff member	Verify an admin can create a staff account and see it listed in personnel management.	High	BLOCKED
Create a product category	Verify an admin can add a new category and see it available for organizing products.	High	FAILED

Waiter Order Management				
Waiter reviews incoming order requests	Validates that a waiter can open the pending request area and see incoming customer requests.	High	PASSED	
Waiter signs in to the demo panel	Validates that a waiter can access the waiter demo panel from the public entry point.	High	FAILED	
Waiter approves a customer session request	Validates that a waiter can approve a pending customer access request and continue the session flow.	High	BLOCKED	
Waiter sees a revoked customer session blocked from access	Validates that a waiter can observe a revoked session state and confirm the blocked access outcome in the panel.	Medium	FAILED	
Waiter handles a real order after authorization	Validates that a waiter can move from an authorized customer session to viewing and processing an actual order.	High	PASSED	
Customer Menu & QR Access				
Reach the customer demo from the homepage feature section	Validates that the homepage demo section also starts the customer-facing menu exploration flow.	Medium	FAILED	
Browse the customer menu successfully	Validates that a customer can open the menu and browse the available items for a table.	High	BLOCKED	
Open customer demo menu from the landing page	Validates that a customer can start the demo flow from the public landing page and reach the menu experience.	High	FAILED	
Retry access after an invalid QR code error	Validates that a customer can use the retry path after an invalid QR code access error and return toward the menu flow.	Medium	FAILED	
Customer Payments				
Customer completes a cash payment with received amount	Verify a customer can complete a cash payment by entering the amount received and submitting the payment request.	High	BLOCKED	
Customer avoids duplicate payment submission	Verify repeated payment submission attempts do not create duplicate payment actions from the customer flow.	Medium	BLOCKED	
Customer requests a payment for an active order	Verify a customer can initiate a payment request from an active order or table session and reach a payment summary flow.	High	PASSED	
Customer sees payment status update after completion	Verify the customer payment status updates from pending to completed within the active session after payment is finalized.	Medium	BLOCKED	

Session Access & Table Claim			
Show table already claimed for a second customer device	Validates that a second customer session cannot take over a table that already has an active claim.	Medium	BLOCKED
Preserve a table claim on refresh and repeated access	Validates that refreshing or re-entering the table session keeps the same active claim without creating a conflicting session.	High	BLOCKED
Recover from blocked customer state after revocation is lifted	Validates that a customer with a blocked session can regain access once the block has been removed.	Medium	BLOCKED
Claim a table and reach ordered access	Validates the customer can start from a table page and reach an authorized session state that allows ordering.	High	BLOCKED
Show pending session while waiting for waiter verification	Validates that a customer session remains pending after requesting access until staff approval occurs.	Medium	BLOCKED

3 Test Execution Breakdown

Failed & Blocked Test Details

Adjust item quantity before ordering

ATTRIBUTES	
Status	BLOCKED
Priority	High
Description	Validates that a customer can update item quantities in the cart and keep the final order summary in sync.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080735-ed18ce39.webm

```

1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.onrender.com",
42         # "input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Müşteri Demo' link to open the customer demo
52         # experience.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
55         # "desc":"Click 'Müşteri Demo QR menüsü' link",
56         # "input":null,"field":"8"}
57         # Müşteri Demo QR menüsünü görüntüleyin. ürünleri... link

```

```
47     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin. Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Tekrar Dene' button to retry access to the
customer demo page
51     # @@ts-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"327"}
52     # Tekrar Dene button
53     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Tekrar Dene' button to retry access to the
customer demo page.
57     # @@ts-step {"i":4,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"337"}
58     # Tekrar Dene button
59     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
60     await elem.click(timeout=10000)
61
62     # --> Assertions to verify final state
63
64     # --> Could not verify the updated item quantity in the cart
because the demo menu is inaccessible due to an expired QR
error.
65     # Assert-outcome: failed
66     # Assert: Expected the access error 'Erişim Hatası' to be
absent so the cart quantity could be verified.
67     await expect(page.locator("xpath=/html/body/section").nth(0)).
to_contain_text("Eri\u015fim Hatas\u0131", timeout=15000),
"Expected the access error 'Eri\u015fim Hatas\u0131' to be
absent so the cart quantity could be verified."
68
69     # --> Could not verify the order summary matches the adjusted
cart because the demo menu is inaccessible due to an expired
QR error.
70     await page.locator("xpath=/html/body/div[2]/div/button").nth
(0).scroll_into_view_if_needed()
71     # Assert-outcome: failed
72     # Assert: Expected the 'Tekrar Dene' button to be absent so
the order summary could be inspected.
73     await expect(page.locator("xpath=/html/body/div[2]/div/
button").nth(0)).to_be_visible(timeout=15000), "Expected the
'Tekrar Dene' button to be absent so the order summary could
be inspected."
74
75     # --> Test blocked by environment/access constraints during
agent run
76     # Reason: TEST BLOCKED The test could not be run – the demo
menu is inaccessible due to an expired QR error shown on the
page. Observations: - The page displays 'Erişim Hatası' with
the message 'Bu QR kod artık geçerli değil. Lütfen işletme
personelinden yeni QR kod isteyin.' and only a 'Tekrar Dene'
button is available. - Clicking the 'Tekrar Dene' button was
```



```

77         attempted twice and the accesserror persist...
       raise AssertionError("Test blocked during agent run: " + "TEST
       BLOCKED The test could not be run \u2014 the demo menu is
       inaccessible due to an expired QR error shown on the page.
       Observations: - The page displays 'Eri\u015fim Hatas\u0131'
       with the message 'Bu QR kod art\u0131k ge\u00e7erli
       de\u011fil. L\u00fctfen i\u015fletme personelinden yeni QR kod
       isteyin.' and only a 'Tekrar Dene' button is available. -
       Clicking the 'Tekrar Dene' button was attempted twice and the
       access error persist..." + " – the exported script cannot
       reproduce a PASS in this environment.")
78     await asyncio.sleep(5)
79
80     finally:
81         if context:
82             await context.close()
83         if browser:
84             await browser.close()
85         if pw:
86             await pw.stop()
87
88     asyncio.run(run_test())
89

```

Error

TEST BLOCKED The test could not be run — the demo menu is inaccessible due to an expired QR error shown on the page. Observations: - The page displays 'Erişim Hatası' with the message 'Bu QR kod artık geçerli değil. Lütfen işletme personelinden yeni QR kod isteyin.' and only a 'Tekrar Dene' button is available. - Clicking the 'Tekrar Dene' button was attempted twice and the access error persisted; the menu and cart controls are not present. - Without a valid QR or any UI path to reach the menu, the cart quantity update flow cannot be exercised.

Cause

The hosting side is likely serving an expired or invalid QR token for the demo menu, causing the app's access gate to reject entry before the menu loads. This can happen if the deployed QR/token has timed out, the server clock is incorrect, the token-signing secret changed between deploys, or the retry flow does not fetch a fresh token and keeps hitting the same expired access state.

Fix

Ensure the demo route/QR token is always valid in the hosted environment (or provide a fallback public demo menu URL that bypasses QR validation for automated tests). Check backend QR expiry logic, seed the deployed environment with a non-expiring/test QR code, and verify the retry button actually refreshes/reloads a valid token instead of reusing an expired one. Also confirm server time/clock, environment variables, and any token-signing secrets on the host are correct so valid QR codes are not being rejected.

Reach the customer demo from the homepage feature section

ATTRIBUTES	
Status	FAILED
Priority	Medium
Description	Validates that the homepage demo section also starts the customer-facing menu exploration flow.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080656-a0147d82.webm

```
1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Müşteri Demo' link labeled 'Müşteri Demo – QR
52         # menüyü görüntüleyin, ürünleri sepete ekleyin ve sipariş
53         # akışını test edin.' to start the customer demo flow.
54         # @@ts-step {"i":2,"type":"action","action":"click",
55         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
56         # "desc":"Click 'Müşteri Demo QR menüyü görüntüleyin, ürünleri sepete ekleyin, sipariş akışını test edin.' link",
57         # "input":null,"field":"8"}
```

```

46     # Müşteri Demo QR menüyü görüntüleyin, ürünleri... link
47     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
    görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
    edin. Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # --> Assertions to verify final state
51
52     # --> Customer demo flow did not open; the page showed an
    access error instead of the customer menu.
53     # Assert-outcome: failed
54     # Assert: Expected the 'Tekrar Dene' button to not be visible
    so the customer demo flow opens successfully.
55     await expect(page.locator("xpath=/html/body/div[2]/div/
    button").nth(0)).not_to_be_visible(timeout=15000), "Expected
    the 'Tekrar Dene' button to not be visible so the customer
    demo flow opens successfully."
56     await asyncio.sleep(5)
57
58     finally:
59         if context:
60             await context.close()
61         if browser:
62             await browser.close()
63         if pw:
64             await pw.stop()
65
66     asyncio.run(run_test())
67

```

Error

TEST FAILURE The customer demo flow did not open successfully — an access error page was shown instead of the customer menu. Observations: - The page displayed the message 'Erişim Hatası' with explanatory text that the QR code is no longer valid. - A 'Tekrar Dene' (Retry) button is visible, and no menu items or product listing are present.

Cause

The hosting-side demo instance is likely using an invalid, missing, or expired QR-code access token/configuration in production. The app is reaching the QR access-validation screen because the backend or frontend cannot confirm a valid menu link for the demo, possibly due to an incorrect environment variable, stale deployment data, or a mismatch between the hosted app URL and the QR/menu identifier stored on the server.

Fix

Verify the production/demo configuration on Render is pointing to the correct live menu/restaurant ID and QR token, and that any environment variables, backend route, or database record used to validate the QR code have been deployed and are reachable. If the demo token expired or the QR validation logic is too strict in production, regenerate/seed a valid QR code mapping or relax the validity check for the customer demo route so the menu page loads instead of the access error page.

Browse the customer menu successfully

ATTRIBUTES	
Status	BLOCKED
Priority	High
Description	Validates that a customer can open the menu and browse the available items for a table.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080722-36b88ec1.webm

```
1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Demoyu Deneyin' button to open the demo role
52         # selection or start the demo flow.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/nav/div[2]/button",
55         # "desc":"Click 'Demoyu Deneyin button'","input":null,
56         # "field":"4"}
57         # Demoyu Deneyin button
58         elem = page.get_by_text('ORMenü', exact=True).locator
```

```

    ("xpath=ancestor-or-self::*[.//button][1]").get_by_role
    ('button', name='Demoyu Deneyin', exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Müşteri Demo' option in the 'Demo Seçin' modal
    to open the customer menu view.
51     # @@ts-step {"i":3,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/div/div/div[2]/a",
    "desc":"Click 'Müşteri Demo QR menüleri... link'",
    "input":null,"field":"345"}
52     # Müşteri Demo QR menüyü görüntüleyin, ürünleri... link
53     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
    görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
    edin.', exact=True)
54     await elem.click(timeout=10000)
55
56     # --> Assertions to verify final state
57
58     # --> Menu content was not displayed because the demo page
    shows an access error instead of the menu.
59     # Assert-outcome: failed
60     # Assert: Expected menu content to be displayed.
61     await expect(page.locator("xpath=/html/body/div[2]/div/
    button").nth(0)).not_to_be_visible(timeout=15000), "Expected
    menu content to be displayed."
62
63     # --> Menu items were not available to browse because the page
    shows an access error ('Erişim Hatası').
64     # Assert-outcome: failed
65     # Assert: Expected menu items to be available to browse.
66     await expect(page.locator("xpath=/html/body/section").nth(0)).
    not_to_be_visible(timeout=15000), "Expected menu items to be
    available to browse."
67
68     # --> Test blocked by environment/access constraints during
    agent run
69     # Reason: TEST BLOCKED The customer menu could not be opened
    because the demo page shows an access error that prevents
    viewing menu content. Observations: - The page title shows
    'Erişim Hatası'. - The message reads: 'Bu QR kod artık geçerli
    değil. Lütfen işletme personelinde yeni QR kod isteyin.' - A
    visible button labeled 'Tekrar Dene' (Retry) is present; no
    menu items or categories are visible to browse.
70     raise AssertionError("Test blocked during agent run: " + "TEST
    BLOCKED The customer menu could not be opened because the demo
    page shows an access error that prevents viewing menu content.
    Observations: - The page title shows 'Erişim Hatası'. - The
    message reads: 'Bu QR kod artık geçerli değil. Lütfen işletme
    personelinde yeni QR kod isteyin.' - A visible button labeled
    'Tekrar Dene' (Retry) is present; no menu items or categories
    are visible to browse." + " - the exported script cannot
    reproduce a PASS in this environment.")
71     await asyncio.sleep(5)
72
73     finally:
74         if context:
75             await context.close()

```

```
76         if browser:
77             await browser.close()
78         if pw:
79             await pw.stop()
80
81     asyncio.run(run_test())
82
```

Error

TEST BLOCKED The customer menu could not be opened because the demo page shows an access error that prevents viewing menu content. Observations: - The page title shows 'Erişim Hatası'. - The message reads: 'Bu QR kod artık geçerli değil. Lütfen işletme personelininden yeni QR kod isteyin.' - A visible button labeled 'Tekrar Dene' (Retry) is present; no menu items or categories are visible to browse.

Cause

The hosting-side service is likely failing to resolve the requested QR code to an active menu because the deployed demo is using an invalid, expired, or missing QR/menu identifier, or it cannot reach the correct backend/database due to misconfigured environment variables or routing. As a result, the app falls back to the access-error page ('Bu QR kod artık geçerli değil').

Fix

Check the hosting-side QR/menu resolution logic and the demo environment configuration. Most likely the deployed app is treating the current demo QR code or menu token as invalid/expired because of a missing or mismatched environment variable, database record, or redirect/rewrite rule. Restore the correct public demo menu mapping, verify the QR code token exists in the backend and is not marked inactive, and ensure the hosting platform is pointing to the right API/database and serving the app's routes correctly. If the issue is caused by an expired demo key or a stale deployment, regenerate and redeploy a valid demo QR/menu link and update any CDN/cache or environment settings so the menu page can load normally.

Submit an order with special instructions

ATTRIBUTES	
Status	BLOCKED
Priority	Medium
Description	Validates that a customer can include notes for an order and submit it through the QR menu flow.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080730-7a8d81b5.webm

```

1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Müşteri Demo' link to open the customer demo
52         # experience.
53         # @ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
55         # "desc":"Click 'Müşteri Demo QR menüsü' link",
56         # "input":null,"field":"8"}
57         # Müşteri Demo QR menüsünü görüntüleyin. ürünleri... link

```

```

47     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin. Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Tekrar Dene' (Retry) button to attempt
reloading the demo menu so the food categories become
available.
51     # @@ts-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button'","input":null,"field":"327"}
52     # Tekrar Dene button
53     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Tekrar Dene' (Retry) button to attempt
reloading the demo menu.
57     # @@ts-step {"i":4,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button'","input":null,"field":"337"}
58     # Tekrar Dene button
59     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
60     await elem.click(timeout=10000)
61
62     # --> Assertions to verify final state
63
64     # --> A successful order confirmation was not displayed
because the demo customer menu was inaccessible.
65     # Assert-outcome: failed
66     # Assert: Expected a successful order confirmation to be
displayed.
67     await expect(page.locator("xpath=/html/body/section").nth(0)).
to_contain_text("Eri\u015fim Hatas\u0131", timeout=15000),
"Expected a successful order confirmation to be displayed."
68
69     # --> The order could not be verified to include the customer
instruction because no cart or order controls loaded.
70     await page.locator("xpath=/html/body/div[2]/div/button").nth
(0).scroll_into_view_if_needed()
71     # Assert-outcome: failed
72     # Assert: Expected the order flow to be available so the
customer instruction could be entered.
73     await expect(page.locator("xpath=/html/body/div[2]/div/
button").nth(0)).to_be_visible(timeout=15000), "Expected the
order flow to be available so the customer instruction could
be entered."
74
75     # --> Test blocked by environment/access constraints during
agent run
76     # Reason: TEST BLOCKED The test could not be run – the demo
customer menu is inaccessible due to an access error on the
demo page. Observations: - The page displays 'Erişim Hatası'
with the message 'Bu QR kod artık geçerli değil. Lütfen
işletme personelinde yeni QR kod isteyin.' - Clicking the
'Tekrar Dene' (Retry) button twice did not load the menu; the
page remained on the access error. - No menu cate...
77     raise AssertionError("Test blocked during agent run: " + "TEST

```

```

    BLOCKED The test could not be run \u2014 the demo customer
    menu is inaccessible due to an access error on the demo page.
    Observations: - The page displays 'Eri\u015fim Hatas\u0131'
    with the message 'Bu QR kod art\u0131k ge\u00e7erli
    de\u011fil. L\u00fctfen i\u015fletme personelinden yeni QR kod
    isteyin.' - Clicking the 'Tekrar Dene' (Retry) button twice
    did not load the menu; the page remained on the access error.
    - No menu cate..." + " - the exported script cannot reproduce
    a PASS in this environment.")
78     await asyncio.sleep(5)
79
80     finally:
81         if context:
82             await context.close()
83         if browser:
84             await browser.close()
85         if pw:
86             await pw.stop()
87
88     asyncio.run(run_test())
89

```

Error

TEST BLOCKED The test could not be run — the demo customer menu is inaccessible due to an access error on the demo page. Observations: - The page displays 'Erişim Hatası' with the message 'Bu QR kod artık geçerli değil. Lütfen işletme personelinden yeni QR kod isteyin.' - Clicking the 'Tekrar Dene' (Retry) button twice did not load the menu; the page remained on the access error. - No menu categories, items, cart, or order controls are visible, so adding an item, entering a special instruction, and submitting an order cannot be performed.

Cause

The hosting side is likely serving an invalid, expired, or missing demo QR-code record, causing the menu-access validation to fail and return the access error. This can happen if the demo tenant was deleted/unpublished, the QR token was rotated or expired, the backend is reading the wrong environment/database, or a CDN/browser cache is preserving an old 'QR kod artık geçerli değil' response. As a result, the page cannot load the menu UI and retry does not recover.

Fix

Ensure the demo menu route resolves to a valid, active QR/menu token in the hosting environment. Check the backend/API that validates QR codes for expired/revoked/default demo codes, confirm the demo record exists in the production database, and remove any stale cached access-denied response at the edge/CDN. If the app relies on environment variables or database seeding, re-seed or reconfigure the demo tenant/menu so the public test URL always points to a live menu. Add a fallback retry path that re-fetches the QR/menu state from the server instead of reusing a cached invalid state.

Browse menu and add items to cart

ATTRIBUTES	
Status	BLOCKED
Priority	High
Description	Validates that a customer can open the QR menu, choose menu items, and add them to the order cart.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080802-9c037f38.webm

```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Demoyu Deneyin' button
52         # @@ts-step {"i":2,"type":"action","action":"click",
53         # "selector":"xpath=/html/body/div[2]/nav/div[2]/button",
54         # "desc":"Click 'Demoyu Deneyin button'","input":null,
55         # "field":"8"}
56         # Demoyu Deneyin button
57         elem = page.get_by_text('QRMenü', exact=True).locator
58         ("xpath=ancestor-or-self::*[.//button][1]").get_by_role
```



```

81      # --> Test blocked by environment/access constraints during
      agent run
82      # Reason: TEST BLOCKED The customer demo menu could not be
      reached – the demo endpoint shows an access error and retries
      did not recover the session. Observations: - The page displays
      'Erişim Hatası' with the message 'Bu QR kod artık geçerli
      değil. Lütfen işletme personelinden yeni QR kod isteyin.'
      indicating the QR/demo session is invalid. - A visible 'Tekrar
      Dene' (Retry) button is present, but clickin...
83      raise AssertionError("Test blocked during agent run: " + "TEST
      BLOCKED The customer demo menu could not be reached \u2014 the
      demo endpoint shows an access error and retries did not
      recover the session. Observations: - The page displays
      'Eri\u015fim Hatas\u0131' with the message 'Bu QR kod
      art\u0131k ge\u00e7erli de\u011fil. L\u00fctfen i\u015fletme
      personelinden yeni QR kod isteyin.' indicating the QR/demo
      session is invalid. - A visible 'Tekrar Dene' (Retry) button
      is present, but clickin..." + " – the exported script cannot
      reproduce a PASS in this environment.")
84      await asyncio.sleep(5)
85
86      finally:
87          if context:
88              await context.close()
89          if browser:
90              await browser.close()
91          if pw:
92              await pw.stop()
93
94      asyncio.run(run_test())
95

```

Error

TEST BLOCKED The customer demo menu could not be reached — the demo endpoint shows an access error and retries did not recover the session. Observations: - The page displays 'Erişim Hatası' with the message 'Bu QR kod artık geçerli değil. Lütfen işletme personelinden yeni QR kod isteyin.' indicating the QR/demo session is invalid. - A visible 'Tekrar Dene' (Retry) button is present, but clicking it did not change the page or load the menu after multiple attempts.

Cause

The hosting service appears to be returning an invalid/expired demo QR session, so the menu route rejects access with 'Bu QR kod artık geçerli değil.' This is commonly caused by a stale hardcoded demo link, a missing or expired session record in the backend, lost in-memory session state after deployment/restart, or misconfigured production environment variables causing the app to validate against the wrong tenant or token store.

Fix

Verify the demo/QR session generation and lookup on the hosting side. The app is likely serving an expired, missing, or mismatched QR token/slug in production, or the backend session store used to validate demo QR codes is not persistent across requests/restarts. Fix by ensuring the demo endpoint points to a valid current menu/session, regenerating the demo QR token, and confirming the hosting environment has stable persistence (database/kv/session storage) and correct environment variables for the demo tenant. Also check that the retry button actually re-requests the active menu/session from the backend and that any cache/CDN is not serving an old invalid token.

Customer completes a cash payment with received amount

ATTRIBUTES	
Status	BLOCKED
Priority	High
Description	Verify a customer can complete a cash payment by entering the amount received and submitting the payment request.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080743-99b6c3f6.webm

```

1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Müşteri Demo' link to open a customer demo
52         # session.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
55         # "desc":"Click 'Müşteri Demo QR menüsünü görüntüleyin, ürünleri... link'",
56         # "input":null,"field":"11"}
57         # Müşteri Demo QR menüsünü görüntüleyin, ürünleri... link

```

```

47     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin. Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the visible 'Tekrar Dene' button to retry opening
the customer/demo session and load the table/payment UI.
51     # @@ts-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"328"}
52     # Tekrar Dene button
53     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Tekrar Dene' button to retry opening the
customer/demo session and load the table/payment UI.
57     # @@ts-step {"i":4,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"338"}
58     # Tekrar Dene button
59     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
60     await elem.click(timeout=10000)
61
62     # --> Assertions to verify final state
63
64     # --> A successful payment confirmation was not shown because
the demo session shows an expired QR error.
65     # Assert-outcome: failed
66     # Assert: Expected a successful payment confirmation to be
displayed.
67     await expect(page.locator("xpath=/html/body/section").nth(0)).
to_contain_text("Eri\u015fim Hatası Bu QR kod art\u0131k
ge\u00e7erli de\u011fil.", timeout=15000), "Expected a
successful payment confirmation to be displayed."
68
69     # --> The order/session could not be marked completed because
no active table session loaded (the Retry button remained
visible and retries did not open a session).
70     await page.locator("xpath=/html/body/div[2]/div/button").nth
(0).scroll_into_view_if_needed()
71     # Assert-outcome: failed
72     # Assert: Expected the order or table session to be reachable
and show the payment as completed.
73     await expect(page.locator("xpath=/html/body/div[2]/div/
button").nth(0)).to_be_visible(timeout=15000), "Expected the
order or table session to be reachable and show the payment as
completed."
74
75     # --> Test blocked by environment/access constraints during
agent run
76     # Reason: TEST BLOCKED The test could not be run – the
customer session could not be reached because the demo QR is
expired. Observations: - The page shows 'Erişim Hatası'
indicating the QR is expired. - Clicking the 'Tekrar Dene'
(Retry) button did not open an active customer session.
77     raise AssertionError("Test blocked during agent run: " + "TEST
BLOCKED The test could not be run \u0141 the customer session

```

could not be reached because the demo QR is expired.
Observations: - The page shows 'Eri\u015fim Hatas\u0131' indicating the QR is expired. - Clicking the 'Tekrar Dene' (Retry) button did not open an active customer session." + " – the exported script cannot reproduce a PASS in this environment.")

```
78     await asyncio.sleep(5)
79
80     finally:
81         if context:
82             await context.close()
83         if browser:
84             await browser.close()
85         if pw:
86             await pw.stop()
87
88     asyncio.run(run_test())
89
```

Error

TEST BLOCKED The test could not be run — the customer session could not be reached because the demo QR is expired. Observations: - The page shows 'Erişim Hatası' indicating the QR is expired. - Clicking the 'Tekrar Dene' (Retry) button did not open an active customer session.

Cause

The hosted demo appears to be serving an expired QR/session token, likely due to a backend session expiry/TTL issue, stale hardcoded demo QR, or a mismatch between the frontend link and the active customer session on the server. The retry button likely points to the same expired token or an endpoint that is failing to create/restore a live session, so the customer session cannot be reached.

Fix

Ensure the QR/demo session generation uses a valid, non-expired token or dynamically refreshes/regenerates the QR link on page load and on retry. Check the hosting configuration and backend logic for session TTL/expiration handling, make sure the demo customer session endpoint is reachable and returning a fresh active session, and verify any environment variables, clock/time sync, or deployment rules that may be causing the QR to be marked expired immediately.

Show table already claimed for a second customer device

ATTRIBUTES	
Status	BLOCKED
Priority	Medium
Description	Validates that a second customer session cannot take over a table that already has an active claim.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080632-544a4119.webm

```
1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Müşteri Demo' link to open the customer QR
52         # menu demo page.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
55         # "desc":"Click 'Müşteri Demo QR menüsü için, ürünleri... link'",
56         # "input":null,"field":"8"}
57         # Müşteri Demo QR menüsünü görüntüleyin, ürünleri... link
```

```

47     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin. Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # --> Assertions to verify final state
51
52     # --> The table/session UI was not shown; the page displays an
access error stating the QR code is invalid.
53     await page.locator("xpath=/html/body/section").nth(0).
scroll_into_view_if_needed()
54     # Assert-outcome: failed
55     # Assert: Expected the notifications section to be visible
indicating the QR code is invalid.
56     await expect(page.locator("xpath=/html/body/section").nth(0)).
to_be_visible(timeout=15000), "Expected the notifications
section to be visible indicating the QR code is invalid."
57
58     # --> Access to ordering is blocked; the page only offers a
'Tekrar Dene' (Retry) button.
59     await page.locator("xpath=/html/body/div[2]/div/button").nth
(0).scroll_into_view_if_needed()
60     # Assert-outcome: failed
61     # Assert: Expected the 'Tekrar Dene' retry button to be
visible, indicating ordering access is blocked.
62     await expect(page.locator("xpath=/html/body/div[2]/div/
button").nth(0)).to_be_visible(timeout=15000), "Expected the
'Tekrar Dene' retry button to be visible, indicating ordering
access is blocked."
63
64     # --> Test blocked by environment/access constraints during
agent run
65     # Reason: TEST BLOCKED The test could not be run because the
required customer table/session page is not reachable – the
demo page reports the QR code is invalid and prevents starting
a session. Observations: - The page displays 'Erişim Hatası'
and the message 'Bu QR kod artık geçerli değil. Lütfen işletme
personelinden yeni QR kod isteyin.' - The only available
action is a 'Tekrar Dene' (Retry) button; ...
66     raise AssertionError("Test blocked during agent run: " + "TEST
BLOCKED The test could not be run because the required
customer table/session page is not reachable \u2014 the demo
page reports the QR code is invalid and prevents starting a
session. Observations: - The page displays 'Eri\u015fim
Hatas\u0131' and the message 'Bu QR kod art\u0131k
ge\u00e7erli de\u011fil. L\u00fctfen i\u015fletme
personelinden yeni QR kod isteyin.' - The only available
action is a 'Tekrar Dene' (Retry) button; ..." + " – the
exported script cannot reproduce a PASS in this environment.")
67     await asyncio.sleep(5)
68
69     finally:
70         if context:
71             await context.close()
72         if browser:
73             await browser.close()
74         if pw:
75             await pw.stop()
76

```

```
77     asyncio.run(run_test())
78
```

Error

TEST BLOCKED The test could not be run because the required customer table/session page is not reachable — the demo page reports the QR code is invalid and prevents starting a session. Observations: - The page displays 'Erişim Hatası' and the message 'Bu QR kod artık geçerli değil. Lütfen işletme personelinden yeni QR kod isteyin.' - The only available action is a 'Tekrar Dene' (Retry) button; no table/session UI or claim controls are present.

Cause

The host is likely serving the app with no active table/session associated with the requested QR code, or the backend lookup for that QR/session is failing due to missing/expired token, misconfigured environment variables, stale database data, or a broken route/API. As a result, the app falls into its invalid-QR error state ('Erişim Hatası') and blocks access to the customer table/session page.

Fix

Ensure the hosted app resolves a valid customer table/session route instead of defaulting to an expired/invalid QR state. Check the production configuration for the QR/session token source, environment variables, and database records for the demo table. If the deployment relies on a signed token or session ID, regenerate/publish a current QR code, seed or restore the corresponding active session on the backend, and verify the route can fetch it from the API. Also confirm that any public landing page or deep link used by the test is mapped to an active customer table view rather than an error/expired-state page.

Review menu categories before ordering

ATTRIBUTES	
Status	BLOCKED
Priority	Low
Description	Validates that a customer can switch between menu categories and still begin an order from the QR menu experience.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080626-05b44cc4.webm

```

1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Müşteri Demo' link to open the customer demo
52         # experience.
53         # @ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
55         # "desc":"Click 'Müşteri Demo QR menüsü' link",
56         # "input":null,"field":"8"}
57         # Müşteri Demo QR menüsünü görüntüleyin. ürünleri... link

```

```

47     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin. Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # --> Assertions to verify final state
51
52     # --> Could not verify the selected item from the second
category because the demo menu is blocked by an access error.
53     await page.locator("xpath=/html/body/div[2]/div/button").nth
(0).scroll_into_view_if_needed()
54     # Assert-outcome: failed
55     # Assert: Expected the menu UI to be available to show the
selected item from the second category.
56     await expect(page.locator("xpath=/html/body/div[2]/div/
button").nth(0)).to_be_visible(timeout=15000), "Expected the
menu UI to be available to show the selected item from the
second category."
57
58     # --> Could not verify that the cart can be used to continue
the order flow because the cart UI is not accessible due to
the access error.
59     await page.locator("xpath=/html/body/div[2]/div/button").nth
(0).scroll_into_view_if_needed()
60     # Assert-outcome: failed
61     # Assert: Expected the cart controls to be available so the
order flow could continue.
62     await expect(page.locator("xpath=/html/body/div[2]/div/
button").nth(0)).to_be_visible(timeout=15000), "Expected the
cart controls to be available so the order flow could continue.
"
63
64     # --> Test blocked by environment/access constraints during
agent run
65     # Reason: TEST BLOCKED The customer demo cannot be reached –
the page shows an access error stating the QR code is no
longer valid and the menu UI is not available. Observations: -
The page displays 'Erişim Hatası' with a message that the QR
code is no longer valid. - Only a 'Tekrar Dene' (Retry) button
is present; no menu categories, items, or cart controls are
visible. - The required UI to switch categ...
66     raise AssertionError("Test blocked during agent run: " + "TEST
BLOCKED The customer demo cannot be reached \u2014 the page
shows an access error stating the QR code is no longer valid
and the menu UI is not available. Observations: - The page
displays 'Eri\u015fim Hatas\u0131' with a message that the QR
code is no longer valid. - Only a 'Tekrar Dene' (Retry) button
is present; no menu categories, items, or cart controls are
visible. - The required UI to switch categ..." + " – the
exported script cannot reproduce a PASS in this environment.")
67     await asyncio.sleep(5)
68
69     finally:
70         if context:
71             await context.close()
72         if browser:
73             await browser.close()
74         if pw:
75             await pw.stop()

```

```
76
77     asyncio.run(run_test())
78
```

Error

TEST BLOCKED The customer demo cannot be reached — the page shows an access error stating the QR code is no longer valid and the menu UI is not available. Observations: - The page displays 'Erişim Hatası' with a message that the QR code is no longer valid. - Only a 'Tekrar Dene' (Retry) button is present; no menu categories, items, or cart controls are visible. - The required UI to switch categories and add items to the cart is not accessible due to this error.

Cause

The hosted demo is likely enforcing QR-code validation on the server or at app startup, and that validation is failing because the QR token/config is expired, missing, misconfigured, or the validation endpoint is unreachable. As a result, the app falls back to an access-error screen instead of rendering the menu UI.

Fix

Check the hosting/environment config for any QR validity gate or token verification step that is failing in production. If the demo depends on a signed/expired QR token, regenerate or remove the expiry check for the customer demo, verify the backend/API endpoint used to validate the QR code is reachable and returning success, and ensure the deployed build has the correct environment variables/secrets/base URL. Also confirm the app is not loading stale cached assets or an outdated production data file that marks the QR as invalid; redeploy with the correct active QR payload and clear any cache/CDN state.

Preserve a table claim on refresh and repeated access

ATTRIBUTES	
Status	BLOCKED
Priority	High
Description	Validates that refreshing or re-entering the table session keeps the same active claim without creating a conflicting session.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080651-9268b27d.webm

```

1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Müşteri Demo' link to open the customer demo
52         # (enter the customer/table session flow).
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
55         # "desc":"Click 'Müşteri Demo QR menüsünü görüntüleyin, ürünleri... link'",
56         # "input":null,"field":"8"}
57         # Müşteri Demo QR menüsünü görüntüleyin, ürünleri... link

```

```

47     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin. Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Tekrar Dene' button on the access error page
to retry opening the table session.
51     # @@@@-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"328"}
52     # Tekrar Dene button
53     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
54     await elem.click(timeout=10000)
55
56     # --> Assertions to verify final state
57
58     # --> An active table session was not preserved after retry;
the page reports an access error instead of restoring the
session.
59     # Assert-outcome: failed
60     # Assert: Expected the active session to be preserved but the
page shows an access error 'Erişim Hatası'.
61     await expect(page.locator("xpath=/html/body/section").nth(0)).
to_contain_text("Eri\u0015fim Hatas\u00131", timeout=15000),
"Expected the active session to be preserved but the page
shows an access error 'Eri\u0015fim Hatas\u00131'."
62
63     # --> No table claim flow was reached and duplicate claim
state could not be checked because the page remained on the
invalid-QR error after retry.
64     # Assert-outcome: failed
65     # Assert: Expected no duplicate table claim state to be
created, but the page shows the QR is invalid after retry.
66     await expect(page.locator("xpath=/html/body/section").nth(0)).
to_contain_text("Bu QR kod art\u00131k ge\u000e7erli de\u0011fil.
", timeout=15000), "Expected no duplicate table claim state to
be created, but the page shows the QR is invalid after retry."
67
68     # --> Test blocked by environment/access constraints during
agent run
69     # Reason: TEST BLOCKED A valid table session could not be
reached – the UI reports the scanned QR is invalid and
retrying did not open a session. Observations: - The page
displays 'Erişim Hatası' with the message 'Bu QR kod artık
geçerli değil. Lütfen işletme personelinden yeni QR kod
isteyin.' - The 'Tekrar Dene' button was clicked to retry, but
the page remained on the same error state and no table ses...
70     raise AssertionError("Test blocked during agent run: " + "TEST
BLOCKED A valid table session could not be reached \u2014 the
UI reports the scanned QR is invalid and retrying did not open
a session. Observations: - The page displays 'Eri\u0015fim
Hatas\u00131' with the message 'Bu QR kod art\u00131k
ge\u000e7erli de\u0011fil. L\u000fctfen i\u0015fletme
personelinden yeni QR kod isteyin.' - The 'Tekrar Dene' button
was clicked to retry, but the page remained on the same error
state and no table ses..." + " – the exported script cannot
reproduce a PASS in this environment.")
71     await asyncio.sleep(5)

```

```
72
73     finally:
74         if context:
75             await context.close()
76         if browser:
77             await browser.close()
78         if pw:
79             await pw.stop()
80
81     asyncio.run(run_test())
82
```

Error

TEST BLOCKED A valid table session could not be reached — the UI reports the scanned QR is invalid and retrying did not open a session. Observations: - The page displays 'Erişim Hatası' with the message 'Bu QR kod artık geçerli değil. Lütfen işletme personelinden yeni QR kod isteyin.' - The 'Tekrar Dene' button was clicked to retry, but the page remained on the same error state and no table session or claim flow was exposed. - No UI elements indicating a valid table session (table selection, claim button, or session token) were present after retry.

Cause

The hosted application is likely resolving the scanned QR to an expired, missing, or mismatched table session. This can happen if the production database does not contain the referenced session, the QR token was generated from another environment/secret and cannot be validated on the host, the session TTL has elapsed, or the retry action is not actually making a fresh request and is stuck on cached invalid-session state. In short, the backend is treating the QR as invalid because the live table-session record or token verification data is not available or not aligned in the hosting environment.

Fix

On the hosting/backend side, ensure the QR code points to a currently valid, server-accepted table session and that the session lookup endpoint is returning a live session for the scanned QR. Check for deployment/environment mismatches where production is serving stale or hardcoded QR/session data, expired session records, or a broken retry path that simply re-renders the invalid state without rechecking the backend. Verify that the session generation/validation logic, database/cache keys, and any environment variables used to sign or resolve QR tokens are correct in the hosted environment. If sessions are meant to be persistent, remove unintended expiration; if they are meant to be rotating, regenerate and redeploy a fresh QR code and confirm the frontend can fetch a valid session after retry.

Open customer demo menu from the landing page

ATTRIBUTES	
Status	FAILED
Priority	High
Description	Validates that a customer can start the demo flow from the public landing page and reach the menu experience.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080818-827c6693.webm

```
1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Demoyu Deneyin' button in the page hero to
52         # open demo role options.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section/div[3]/div[2]/
55         # button","desc":"Click 'Demoyu Deneyin button'","input":null,
56         # "field":"10"}
57         # Demoyu Deneyin button
58         elem = page.get_by_text('Restoran ve Kafeler için')
```

```

        Geliştirildi', exact=True).locator("xpath=ancestor-or-self::*
        [./button][1]").get_by_role('button', name='Demoyu Deneyin',
        exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Müşteri Demo' option in the 'Demo Seçin' modal
        to open the customer menu experience.
51     # @@ts-step {"i":3,"type":"action","action":"click",
        "selector":"xpath=/html/body/div[2]/div/div/div[2]/a",
        "desc":"Click 'M\u00fc\u015fteri Demo QR men\u00fcy\u00fc
        g\u00f6r\u00fcnt\u00fcleyin, \u00f6r\u00fcnleri... link'",
        "input":null,"field":"344"}
52     # Müşteri Demo QR menüyü görüntüleyin, ürünleri... link
53     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
        görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
        edin.', exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Tekrar Dene' button to retry loading the
        customer menu experience.
57     # @@ts-step {"i":4,"type":"action","action":"click",
        "selector":"xpath=/html/body/div[2]/div/button","desc":"Click
        'Tekrar Dene button',"input":null,"field":"379"}
58     # Tekrar Dene button
59     elem = page.get_by_role('button', name='Tekrar Dene',
        exact=True)
60     await elem.click(timeout=10000)
61
62     # -> Click the 'Tekrar Dene' button to retry loading the
        customer menu experience.
63     # @@ts-step {"i":5,"type":"action","action":"click",
        "selector":"xpath=/html/body/div[2]/div/button","desc":"Click
        'Tekrar Dene button',"input":null,"field":"389"}
64     # Tekrar Dene button
65     elem = page.get_by_role('button', name='Tekrar Dene',
        exact=True)
66     await elem.click(timeout=10000)
67
68     # --> Assertions to verify final state
69
70     # --> The customer menu experience did not load; an access
        error notification was shown instead.
71     # Assert-outcome: failed
72     # Assert: Expected the access error notification to not be
        visible so the customer menu could load.
73     await expect(page.locator("xpath=/html/body/section").nth(0)).
        not_to_be_visible(timeout=15000), "Expected the access error
        notification to not be visible so the customer menu could load.
        "
74     await asyncio.sleep(5)
75
76     finally:
77         if context:
78             await context.close()
79         if browser:
80             await browser.close()
81         if pw:
82             await pw.stop()
83

```

```
84     asyncio.run(run_test())
85
```

Error

TEST FAILURE The customer demo flow did not reach the customer menu — an access error page appears instead. Observations: - After selecting 'Müşteri Demo', the page navigated to /menu/demo-business-id/1 and displayed 'Erişim Hatası' with the message 'Bu QR kod artık geçerli değil.' - The visible 'Tekrar Dene' button was clicked twice but the error remained and the customer menu did not load. Conclusion: The demo entry path from the public landing page does not allow a customer to reach the menu experience due to a persistent access error. Marking the test as failed (success=false).

Cause

The customer demo link appears to resolve to /menu/demo-business-id/1, but the server treats that QR/menu as invalid and returns an access error ('Bu QR kod artık geçerli değil.'). This suggests a hosting-side data/configuration issue: the demo QR code entry is missing, expired, inactive, or the app is connected to the wrong environment/database after deployment. As a result, the public landing page sends users to a protected route that immediately fails access validation, preventing the menu from loading.

Fix

On the hosting/app backend, ensure the public demo route maps to a valid, active QR/menu identifier and that the demo business/QR record is seeded in the production database. The access check likely rejects the demo slug/id as expired, inactive, or soft-deleted, so update the demo endpoint to bypass normal expiration rules for demo traffic or create a dedicated always-valid demo QR/menu entry. Also verify environment variables and database migrations on Render so the deployed instance points to the correct production data and not a stale demo record.

Waiter signs in to the demo panel

ATTRIBUTES	
Status	FAILED
Priority	High
Description	Validates that a waiter can access the waiter demo panel from the public entry point.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080911-1e89f557.webm

```
1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45
46         try:
47             await page.wait_for_load_state("domcontentloaded",
48                 timeout=5000)
49         except Exception:
50             pass
51
52         # -> Click the 'Demoyu Deneyin' button to open the demo role
53         # selection.
54         # @@ts-step {"i":2,"type":"action","action":"click",
55         # "selector":"xpath=/html/body/div[2]/section/div[3]/div[2]/
56         # button","desc":"Click 'Demoyu Deneyin button'", "input":null,
57         # "field":"9"}
58         # Demoyu Deneyin button
59         elem = page.get_by_text('Restoran ve Kafeler için')
```

```

Geliştirildi', exact=True).locator("xpath=ancestor-or-self::*
[.//button][1]").get_by_role('button', name='Demoyu Deneyin',
exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Garson Demo' option in the "Demo Seçin" modal
to open the waiter demo account selection.
51     # @ts-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/div/div[2]/a[2]",
"desc":"Click 'Garson Demo Gelen sipari\u015fleri
y\u00f6netin, masa... link'", "input":null, "field":"346"}
52     # Garson Demo Gelen siparişleri yönetin, masa... link
53     elem = page.get_by_role('link', name='Garson Demo Gelen
siparişleri yönetin, masa durumlarını takip edin ve ödeme
taleplerini görün.', exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Garson · garson@demo.com' demo account in the
Demo Hesapları list, then click the 'Giriş Yap' button to sign
in and open the waiter dashboard.
57     # @ts-step {"i":4,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[3]/div[3]/div/button[2]
", "desc":"Click 'Garson \u00b7 garson@demo.com button'",
"input":null, "field":"424"}
58     # Garson · garson@demo.com button
59     elem = page.get_by_role('button', name='Garson · garson@demo.
com', exact=True)
60     await elem.click(timeout=10000)
61
62     # -> Click the 'Garson · garson@demo.com' demo account in the
Demo Hesapları list, then click the 'Giriş Yap' button to sign
in and open the waiter dashboard.
63     # @ts-step {"i":5,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/button",
"desc":"Click 'Giri\u015f Yap button'", "input":null,
"field":"408"}
64     # Giriş Yap button
65     elem = page.get_by_role('button', name='Giriş Yap', exact=True)
66     await elem.click(timeout=10000)
67
68     # -> Click the 'Garson · garson@demo.com' demo account, then
click the 'Giriş Yap' button to sign in and open the waiter
dashboard.
69     # @ts-step {"i":6,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[3]/div[3]/div/button[2]
", "desc":"Click 'Garson \u00b7 garson@demo.com button'",
"input":null, "field":"424"}
70     # Garson · garson@demo.com button
71     elem = page.get_by_role('button', name='Garson · garson@demo.
com', exact=True)
72     await elem.click(timeout=10000)
73
74     # -> Click the 'Garson · garson@demo.com' demo account, then
click the 'Giriş Yap' button to sign in and open the waiter
dashboard.
75     # @ts-step {"i":7,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/button",
"desc":"Click 'Giri\u015f Yap button'", "input":null,
"field":"408"}

```

```

76     # Giriş Yap button
77     elem = page.get_by_role('button', name='Giriş Yap', exact=True)
78     await elem.click(timeout=10000)
79
80     # --> Assertions to verify final state
81
82     # --> The waiter dashboard did not open; the app remained on
83     the sign-in page.
84     # Assert-outcome: failed
85     # Assert: Expected the app to navigate to the waiter dashboard
86     but the URL remained on the sign-in page.
87     await expect(page).to_have_url(re.compile("auth/signin\\?
88     demo=waiter"), timeout=15000), "Expected the app to navigate
89     to the waiter dashboard but the URL remained on the sign-in
90     page."
91
92     # --> The sign-in page displayed an error notification after
93     the demo sign-in attempt.
94     await page.locator("xpath=/html/body/section").nth(0).
95     scroll_into_view_if_needed()
96     # Assert-outcome: failed
97     # Assert: Expected the sign-in error notification to be
98     visible.
99     await expect(page.locator("xpath=/html/body/section").nth(0)).
100     to_be_visible(timeout=15000), "Expected the sign-in error
101     notification to be visible."
102
103     # --> The demo waiter email was prefilled in the sign-in form
104     (demo credentials present).
105     # Assert-outcome: failed
106     # Assert: Expected the email input to be prefilled with the
107     demo waiter email.
108     await expect(page.locator("xpath=/html/body/div[2]/div[3]/div
109     [2]/form/div[1]/div/input").nth(0)).to_have_value("garson@demo.
110     com", timeout=15000), "Expected the email input to be
111     prefilled with the demo waiter email."
112     await asyncio.sleep(5)
113
114     finally:
115         if context:
116             await context.close()
117         if browser:
118             await browser.close()
119         if pw:
120             await pw.stop()
121
122     asyncio.run(run_test())
123

```

Error

TEST FAILURE The waiter demo sign-in did not succeed — the demo credentials were not accepted and the waiter dashboard did not load. Observations: - The login page showed the error: 'E-posta veya şifre hatalı. Lütfen bilgilerinizi kontrol edin.' - The page remained on the sign-in screen after selecting 'Garson · garson@demo.com' and clicking 'Giriş Yap'. - The 'Demo Hesapları' list displays 'Garson · garson@demo.com', indicating the demo account is present but sign-in fails.

Cause

The waiter demo account is likely only displayed in the frontend as a sample credential, but the hosted backend/database does not contain a matching active user record or cannot validate it because of an environment mismatch (wrong DB, missing seed data, incorrect password hash/secret, or auth API misconfiguration). As a result, selecting 'Garson · garson@demo.com' submits credentials that the live service rejects, producing 'E-posta veya şifre hatalı' and keeping the user on the sign-in screen.

Fix

Check the hosting/deployment environment for the demo waiter account seed and authentication backend configuration. Most likely the demo account exists in the UI list but is not actually present in the production database, has a different password hash than the one the frontend expects, or the auth request is hitting a misconfigured API/DB on Render (wrong environment variables, missing migration/seed on startup, or a stale build). Re-seed/create the account in the live database with the exact credentials shown in the UI, verify the login endpoint is pointing to the correct database and auth service, and confirm the demo password hash algorithm and environment secrets match across local and hosted environments.

Waiter approves a customer session request

ATTRIBUTES	
Status	BLOCKED
Priority	High
Description	Validates that a waiter can approve a pending customer access request and continue the session flow.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080951-2d462a50.webm

```
1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Garson Demo' link to open the waiter (Garson)
52         # panel.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a[2]",
55         # "desc":"Click 'Garson Demo Gelen sipari\u015fleri
56         # y\u00f6netin, masa... link'", "input":null, "field":"9"}
57         # Garson Demo Gelen siparişleri yönetin, masa... link
58         elem = page.locator('a[href="/auth/signin?demo=waiter"]')
```

```

48     await elem.click(timeout=10000)
49
50     # -> Click the 'Giriş Yap' button to sign in with the Garson
demo account and open the waiter panel.
51     # @@ts-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/button",
"desc":"Click 'Giriş Yap button'", "input":null,
"field":"354"}
52     # Giriş Yap button
53     elem = page.get_by_role('button', name='Giriş Yap', exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Garson · garson@demo.com' demo account button,
then click the 'Giriş Yap' button to sign in and reach the
waiter dashboard.
57     # @@ts-step {"i":4,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[3]/div[3]/div/button[2]
", "desc":"Click 'Garson · garson@demo.com button'",
"input":null, "field":"370"}
58     # Garson · garson@demo.com button
59     elem = page.get_by_role('button', name='Garson · garson@demo.
com', exact=True)
60     await elem.click(timeout=10000)
61
62     # -> Click the 'Garson · garson@demo.com' demo account button,
then click the 'Giriş Yap' button to sign in and reach the
waiter dashboard.
63     # @@ts-step {"i":5,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/button",
"desc":"Click 'Giriş Yap button'", "input":null,
"field":"354"}
64     # Giriş Yap button
65     elem = page.get_by_role('button', name='Giriş Yap', exact=True)
66     await elem.click(timeout=10000)
67
68     # -> Fill the 'Şifre' (password) field with 'password123' and
click the 'Giriş Yap' button to attempt signing in as the
Garson demo account.
69     # @@ts-step {"i":6,"type":"action","action":"fill",
"selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/div[2]/
div/input", "desc":"Fill 'password123' into
\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022 password
field", "input":"password123", "field":"345"}
70     # ..... password field
71     elem = page.locator('[id="şifre"]')
72     await elem.wait_for(state="visible", timeout=10000)
73     await elem.fill("password123")
74
75     # -> Fill the 'Şifre' (password) field with 'password123' and
click the 'Giriş Yap' button to attempt signing in as the
Garson demo account.
76     # @@ts-step {"i":7,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/button",
"desc":"Click 'Giriş Yap button'", "input":null,
"field":"354"}
77     # Giriş Yap button
78     elem = page.get_by_role('button', name='Giriş Yap', exact=True)
79     await elem.click(timeout=10000)
80

```

```

81 # -> Fill 'example@gmail.com' into the E-posta adresi field
    and 'password123' into the Şifre field, then click the 'Giriş
    Yap' button.
82 # @@ts-step {"i":8,"type":"action","action":"fill",
    "selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/div/div/
    input","desc":"Fill 'example@gmail.com' into ornek@isletme.com
    email field","input":"example@gmail.com","field":"333"}
83 # ornek@isletme.com email field
84 elem = page.locator('[id="e-posta-adresi"]')
85 await elem.wait_for(state="visible", timeout=10000)
86 await elem.fill("example@gmail.com")
87
88 # -> Fill 'example@gmail.com' into the E-posta adresi field
    and 'password123' into the Şifre field, then click the 'Giriş
    Yap' button.
89 # @@ts-step {"i":9,"type":"action","action":"fill",
    "selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/div[2]/
    div/input","desc":"Fill 'password123' into
    \u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022 password
    field","input":"password123","field":"345"}
90 # ..... password field
91 elem = page.locator('[id="şifre"]')
92 await elem.wait_for(state="visible", timeout=10000)
93 await elem.fill("password123")
94
95 # -> Fill 'example@gmail.com' into the E-posta adresi field
    and 'password123' into the Şifre field, then click the 'Giriş
    Yap' button.
96 # @@ts-step {"i":10,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/button",
    "desc":"Click 'Giriş Yap button'", "input":null,
    "field":"354"}
97 # Giriş Yap button
98 elem = page.get_by_role('button', name='Giriş Yap', exact=True)
99 await elem.click(timeout=10000)
100
101 # -> Click the 'Garson · garson@demo.com' demo account button,
    then click the 'Giriş Yap' button to attempt signing in to the
    waiter panel.
102 # @@ts-step {"i":11,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/div[3]/div[3]/div/button[2]
    ","desc":"Click 'Garson \u00b7 garson@demo.com button'",
    "input":null,"field":"370"}
103 # Garson · garson@demo.com button
104 elem = page.get_by_role('button', name='Garson · garson@demo.
    com', exact=True)
105 await elem.click(timeout=10000)
106
107 # -> Click the 'Garson · garson@demo.com' demo account button,
    then click the 'Giriş Yap' button to attempt signing in to the
    waiter panel.
108 # @@ts-step {"i":12,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/button",
    "desc":"Click 'Giriş Yap button'", "input":null,
    "field":"354"}
109 # Giriş Yap button
110 elem = page.get_by_role('button', name='Giriş Yap', exact=True)
111 await elem.click(timeout=10000)
112

```

```

113     # --> Assertions to verify final state
114
115     # --> Customer session was not authorized because sign-in
116     # failed with an authentication error.
117     # Assert-outcome: failed
118     # Assert: Expected the notifications area to contain the
119     # authentication error message.
120     await expect(page.locator("xpath=/html/body/section").nth(0)).
121     to_contain_text("E-posta veya \u015fifre hatal\u0131.
122     L\u00fctfen bilgilerinizi kontrol edin.", timeout=15000),
123     "Expected the notifications area to contain the authentication
124     error message."
125
126     # --> Test blocked by environment/access constraints during
127     # agent run
128     # Reason: TEST BLOCKED The waiter sign-in could not be
129     # completed – the demo credentials do not authenticate and no
130     # alternative working waiter credentials are available, so the
131     # test cannot continue to approve a customer request.
132     # Observations: - The page displays the error 'E-posta veya
133     # \u015fifre hatal\u0131. L\u00fctfen bilgilerinizi kontrol edin.' after
134     # sign-in attempts. - Clicking the 'Garson \u00b7 garson@demo.com'
135     # demo ...
136     raise AssertionError("Test blocked during agent run: " + "TEST
137     BLOCKED The waiter sign-in could not be completed \u2014 the
138     demo credentials do not authenticate and no alternative
139     working waiter credentials are available, so the test cannot
140     continue to approve a customer request. Observations: - The
141     page displays the error 'E-posta veya \u015fifre hatal\u0131.
142     L\u00fctfen bilgilerinizi kontrol edin.' after sign-in
143     attempts. - Clicking the 'Garson \u00b7 garson@demo.com'
144     demo ..." + " – the exported script cannot reproduce a PASS in
145     this environment.")
146
147     await asyncio.sleep(5)
148
149     finally:
150         if context:
151             await context.close()
152         if browser:
153             await browser.close()
154         if pw:
155             await pw.stop()
156
157     asyncio.run(run_test())
158
159

```

Error

TEST BLOCKED The waiter sign-in could not be completed — the demo credentials do not authenticate and no alternative working waiter credentials are available, so the test cannot continue to approve a customer request.

Observations: - The page displays the error 'E-posta veya \u015fifre hatal\u0131. L\u00fctfen bilgilerinizi kontrol edin.' after sign-in attempts. - Clicking the 'Garson \u00b7 garson@demo.com' demo account and pressing the 'Giri\u015f Yap' button did not sign in and the signin form remained visible. - Entering fallback credentials (example@gmail.com / password123) produced the same authentication error.

Cause

Most likely the hosted production instance has a broken authentication setup rather than a UI issue: the waiter demo account is missing, not seeded, or its credentials do not match the deployed backend, or the frontend is calling an incorrect/misconfigured auth service. Another common hosting-side cause is missing/incorrect environment variables (database URL, JWT/session secret, auth provider keys) leading the sign-in request to fail even with valid demo credentials.

Fix

Check the production auth configuration on the hosting side: verify the deployed environment variables for the login API/DB/auth provider are present and correct, ensure the demo waiter account exists in the production database with the expected password hash/role, and confirm the frontend is pointing to the correct backend endpoint (not a stale or wrong URL). If the app relies on seeded demo users, rerun the seed/migration process in production. Also inspect server logs for authentication errors (bad JWT secret, missing DB connection, auth provider misconfiguration, CORS/session cookie issues) and redeploy after correcting the environment so the waiter demo credentials authenticate successfully.

Recover from blocked customer state after revocation is lifted

ATTRIBUTES	
Status	BLOCKED
Priority	Medium
Description	Validates that a customer with a blocked session can regain access once the block has been removed.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080803-4cfd3536.webm


```

1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Müşteri Demo' link to open the customer demo
52         # flow.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
55         # "desc":"Click 'Müşteri Demo QR menüsünü görüntüleyin, ürünleri... link'",
56         # "input":null,"field":"11"}
57         # Müşteri Demo QR menüsünü görüntüleyin, ürünleri... link

```

```

47     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin. Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Tekrar Dene' button to retry accessing the
table/session and observe whether access is restored or the
blocked state persists.
51     # @@ts-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button'","input":null,"field":"326"}
52     # Tekrar Dene button
53     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Tekrar Dene' button to re-attempt accessing
the table session.
57     # @@ts-step {"i":4,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button'","input":null,"field":"336"}
58     # Tekrar Dene button
59     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
60     await elem.click(timeout=10000)
61
62     # --> Assertions to verify final state
63
64     # --> The customer did not reach an access-allowed session –
the page shows an access error.
65     # Assert-outcome: failed
66     # Assert: Expected the notifications section to not contain
the access error message 'Erişim Hatası'.
67     await expect(page.locator("xpath=/html/body/section").nth(0)).
to_contain_text("Eri\u015fim Hatas\u0131", timeout=15000),
"Expected the notifications section to not contain the access
error message 'Eri\u015fim Hatas\u0131'."
68
69     # --> Ordering access was not restored – the 'Tekrar Dene'
(Retry) button remains visible after attempts to retry.
70     # Assert-outcome: failed
71     # Assert: Expected the 'Tekrar Dene' button to not be visible
after retrying to restore access.
72     await expect(page.locator("xpath=/html/body/div[2]/div/
button").nth(0)).not_to_be_visible(timeout=15000), "Expected
the 'Tekrar Dene' button to not be visible after retrying to
restore access."
73
74     # --> Test blocked by environment/access constraints during
agent run
75     # Reason: TEST BLOCKED The test could not be run – the UI
provides no way to remove the customer's blocked session from
the customer-facing page. Observations: - The page shows an
'Erişim Hatası' (Access Error) message telling the user the QR
code is no longer valid. - The only actionable control on the
page is the 'Tekrar Dene' (Retry) button; it was clicked twice
and did not restore access. - No admin/...
76     raise AssertionError("Test blocked during agent run: " + "TEST
BLOCKED The test could not be run \u2014 the UI provides no

```

```

way to remove the customer's blocked session from the
customer-facing page. Observations: - The page shows an
'Eri\u015fim Hatas\u0131' (Access Error) message telling the
user the QR code is no longer valid. - The only actionable
control on the page is the 'Tekrar Dene' (Retry) button; it
was clicked twice and did not restore access. - No admin/...
+ " - the exported script cannot reproduce a PASS in this
environment.")
77     await asyncio.sleep(5)
78
79     finally:
80         if context:
81             await context.close()
82         if browser:
83             await browser.close()
84         if pw:
85             await pw.stop()
86
87     asyncio.run(run_test())
88

```

Error

TEST BLOCKED The test could not be run — the UI provides no way to remove the customer's blocked session from the customer-facing page. Observations: - The page shows an 'Erişim Hatası' (Access Error) message telling the user the QR code is no longer valid. - The only actionable control on the page is the 'Tekrar Dene' (Retry) button; it was clicked twice and did not restore access. - No admin/demo unblock controls, contact link, or alternate navigation to lift the block are visible on this page.

Cause

The hosted app is likely treating the customer session or QR token as invalid/expired due to a server-side access-control or configuration issue (for example a bad environment variable, expired token, mismatched deployment state, or unreachable session store). Because the public page only offers a retry that reuses the same blocked state and exposes no unblock/reset path, the session becomes unrecoverable from the customer-facing UI.

Fix

Check the hosting/runtime configuration for any environment-driven access gate or QR-session validation that is marking the public route as expired/blocked. Ensure the deployed frontend can load the correct production API/config and that the backend session/token store is reachable and not returning stale/invalid state. Add a recoverable fallback on the customer page (e.g. re-scan/new QR flow, support/contact link, or demo reset endpoint) so a blocked session can be cleared without admin access, and verify any CDN/proxy cache is not serving an expired QR status page indefinitely.

Claim a table and reach ordered access

ATTRIBUTES	
Status	BLOCKED
Priority	High
Description	Validates the customer can start from a table page and reach an authorized session state that allows ordering.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080745-f5f33ccd.webm

```

1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Müşteri Demo' link to open the customer (table/
52         # QR) demo flow.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
55         # "desc":"Click 'Müşteri Demo QR menüsü için ürünleri... link'",
56         # "input":null,"field":"11"}
57         # Müşteri Demo QR menüsünü görüntüleyin, ürünleri... link

```

```

47     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin. Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Tekrar Dene' (Retry) button to attempt to
reopen the table session.
51     # @@ts-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"327"}
52     # Tekrar Dene button
53     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Tekrar Dene' button to retry opening the table
session and observe whether the page transitions to an
authorized ordering state.
57     # @@ts-step {"i":4,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"337"}
58     # Tekrar Dene button
59     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
60     await elem.click(timeout=10000)
61
62     # --> Assertions to verify final state
63
64     # --> The session did not reach an authorized ordering state
because the page showed an access error about an expired/
invalid QR.
65     # Assert-outcome: failed
66     # Assert: Expected the session to reach an authorized ordering
state.
67     await expect(page.locator("xpath=/html/body/section").nth(0)).
to_contain_text("Erişim Hatası Bu QR kod artıkk
geçerli değildir. Lütfen işletme
personelinden yeni QR kod isteyin.", timeout=15000), "Expected
the session to reach an authorized ordering state."
68
69     # --> The retry control 'Tekrar Dene' remained visible and did
not transition the page to an authorized session after being
clicked.
70     await page.locator("xpath=/html/body/div[2]/div/button").nth
(0).scroll_into_view_if_needed()
71     # Assert-outcome: failed
72     # Assert: Expected the 'Tekrar Dene' button to start or
restore an authorized ordering session when clicked.
73     await expect(page.locator("xpath=/html/body/div[2]/div/
button").nth(0)).to_be_visible(timeout=15000), "Expected the
'Tekrar Dene' button to start or restore an authorized
ordering session when clicked."
74
75     # --> Test blocked by environment/access constraints during
agent run
76     # Reason: TEST BLOCKED The customer table session could not be
started because the demo QR appears to be expired or invalid
and the UI did not allow proceeding to an authorized session.
Observations: - The page displays 'Erişim Hatası' with the

```

```

77         message 'Bu QR kod artık geçerli değil. Lütfen işletme
           personelinden yeni QR kod isteyin.' - The visible 'Tekrar
           Dene' button was clicked twice and did not transi...
78         raise AssertionError("Test blocked during agent run: " + "TEST
           BLOCKED The customer table session could not be started
           because the demo QR appears to be expired or invalid and the
           UI did not allow proceeding to an authorized session.
           Observations: - The page displays 'Eri\u015fim Hatas\u0131'
           with the message 'Bu QR kod artık geçerli değil. Lütfen işletme
           personelinden yeni QR kod isteyin.' - The visible 'Tekrar Dene'
           button was clicked twice and did not transi..." + " - the
           exported script cannot reproduce a PASS in this environment.")
79         await asyncio.sleep(5)
80     finally:
81         if context:
82             await context.close()
83         if browser:
84             await browser.close()
85         if pw:
86             await pw.stop()
87
88     asyncio.run(run_test())
89

```

Error

TEST BLOCKED The customer table session could not be started because the demo QR appears to be expired or invalid and the UI did not allow proceeding to an authorized session. Observations: - The page displays 'Erişim Hatası' with the message 'Bu QR kod artık geçerli değil. Lütfen işletme personelinden yeni QR kod isteyin.' - The visible 'Tekrar Dene' button was clicked twice and did not transition the page to an ordering/authorized session state - No alternative in-page navigation or start-session control is available to obtain a valid table session from this screen

Cause

The hosted app is likely serving a stale or expired demo QR/session token, or the server-side QR validation is rejecting the token because of a configuration mismatch (for example an outdated secret, wrong environment variable, bad timestamp/expiry logic, or stale database entry). As a result, the page correctly shows an access error and the visible retry action cannot recover because it keeps reusing the same invalid QR/session state instead of obtaining a new authorized table session.

Fix

Fix the hosting/app flow so the demo landing page always has a valid, non-expired table session available for automated testing. Concretely: verify the QR/session token generation and validation logic on the server, ensure the deployed environment is using the correct current token/secret and not an old hardcoded QR payload, extend or disable expiry for the demo route, and make the Retry button re-fetch a fresh session or redirect to a public start/demo session page instead of reloading the same invalid state. Also check that any environment variables, database records, or time-based signing keys used to validate the QR are correctly configured in production and synchronized with the deployment.

Show pending session while waiting for waiter verification

ATTRIBUTES	
Status	BLOCKED
Priority	Medium
Description	Validates that a customer session remains pending after requesting access until staff approval occurs.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080813-a726cda5.webm


```

1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Müşteri Demo' link to open the customer/table
52         # demo session page.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
55         # "desc":"Click 'M\u00fcs\u00fcter\u00fc Demo QR men\u00fc\u00fcs\u00fc
56         # g\u00fc\u00fcs\u00fcnt\u00fcleyin, \u00fcs\u00fcs\u00fcnt\u00fcleyin... link'",
57         # "input":null,"field":"8"}
58         # Müşteri Demo QR menüvü görüntüleyin, ürünleri... link

```

```

47     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin. Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Tekrar Dene' (Try Again) button to retry
opening the customer/table demo session.
51     # @@ts-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"328"}
52     # Tekrar Dene button
53     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Tekrar Dene' button to retry opening the
customer/table demo session.
57     # @@ts-step {"i":4,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"338"}
58     # Tekrar Dene button
59     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
60     await elem.click(timeout=10000)
61
62     # -> Click the 'Tekrar Dene' button to retry opening the
customer/table demo session and observe whether the session
flow becomes available.
63     # @@ts-step {"i":5,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"395"}
64     # Tekrar Dene button
65     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
66     await elem.click(timeout=10000)
67
68     # --> Assertions to verify final state
69
70     # --> The session is not shown as pending approval; the page
reports the QR code is invalid.
71     # Assert-outcome: failed
72     # Assert: Expected the session to be shown as pending approval.
73     await expect(page.locator("xpath=/html/body/section").nth(0)).
to_contain_text("Bu QR kod art\u0131k ge\u00e7erli de\u011fil.",
timeout=15000), "Expected the session to be shown as
pending approval."
74
75     # --> Ordering access was not enabled because the demo QR/
session is invalid and only a retry button is available.
76     await page.locator("xpath=/html/body/div[2]/div/button").nth
(0).scroll_into_view_if_needed()
77     # Assert-outcome: failed
78     # Assert: Expected ordering access to be not yet fully enabled.
79     await expect(page.locator("xpath=/html/body/div[2]/div/
button").nth(0)).to_be_visible(timeout=15000), "Expected
ordering access to be not yet fully enabled."
80
81     # --> Test blocked by environment/access constraints during
agent run

```

```

82      # Reason: TEST BLOCKED A valid customer/table session could
      not be started because the demo QR is invalid and the UI
      provides no path to request access. Observations: - The page
      displays 'Erişim Hatası' with the message 'Bu QR kod artık
      geçerli değil.' - Only actionable control on the page is the
      'Tekrar Dene' button; clicking it (3 attempts) did not change
      the error state. - No form or controls are avai...
83      raise AssertionError("Test blocked during agent run: " + "TEST
      BLOCKED A valid customer/table session could not be started
      because the demo QR is invalid and the UI provides no path to
      request access. Observations: - The page displays 'Eri\u015fim
      Hatas\u0131' with the message 'Bu QR kod art\u0131k
      ge\u00e7erli de\u011fil.' - Only actionable control on the
      page is the 'Tekrar Dene' button; clicking it (3 attempts) did
      not change the error state. - No form or controls are avai..."
      + " - the exported script cannot reproduce a PASS in this
      environment.")
84      await asyncio.sleep(5)
85
86      finally:
87          if context:
88              await context.close()
89          if browser:
90              await browser.close()
91          if pw:
92              await pw.stop()
93
94      asyncio.run(run_test())
95

```

Error

TEST BLOCKED A valid customer/table session could not be started because the demo QR is invalid and the UI provides no path to request access. Observations: - The page displays 'Erişim Hatası' with the message 'Bu QR kod artık geçerli değil.' - Only actionable control on the page is the 'Tekrar Dene' button; clicking it (3 attempts) did not change the error state. - No form or controls are available on the page to start a table claim or submit an access request, so the pending-approval flow cannot be exercised.

Cause

The hosted app is likely using an expired or misconfigured QR/session token in production, causing the QR validation endpoint to reject the demo code and render the 'Bu QR kod artık geçerli değil.' state. Another likely cause is a backend/config mismatch on the hosting side (wrong API base URL, stale environment variables, incorrect signature secret, or missing demo table record) so the app cannot initialize a customer/table session. Because the UI lacks any alternate access-request path, the failure becomes a hard block instead of a recoverable pending-approval flow.

Fix

Restore a valid demo QR/session bootstrap path on the hosting side. Ensure the deployed environment serves a live, non-expired QR token or a fallback demo table/session ID, and verify any QR validation logic is pointed at the correct backend/config. If the intended flow is pending approval, add a visible request-access/table-claim form or CTA when the QR is invalid so users can still submit a session/access request. Also check that environment variables, signing keys, table IDs, and API endpoints used to validate QR codes are correctly configured in production and not returning stale/expired demo data.

Customer avoids duplicate payment submission

ATTRIBUTES	
Status	BLOCKED
Priority	Medium
Description	Verify repeated payment submission attempts do not create duplicate payment actions from the customer flow.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080757-5b898a0c.webm

```
1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Müşteri Demo' link to start a customer demo
52         # session.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
55         # "desc":"Click 'Müşteri Demo QR menüsünü görüntüleyin, ürünleri... link'",
56         # "input":null,"field":"11"}
57         # Müşteri Demo QR menüsünü görüntüleyin, ürünleri... link
```

```

47     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin. Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Tekrar Dene' button to retry opening the
customer demo session.
51     # @@ts-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"328"}
52     # Tekrar Dene button
53     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Tekrar Dene' button to retry opening the
customer demo session.
57     # @@ts-step {"i":4,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"338"}
58     # Tekrar Dene button
59     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
60     await elem.click(timeout=10000)
61
62     # --> Assertions to verify final state
63
64     # --> Payment flow could not be completed because the customer
demo session was blocked by an expired QR code.
65     await page.locator("xpath=/html/body/div[2]/div/button").nth
(0).scroll_into_view_if_needed()
66     # Assert-outcome: failed
67     # Assert: Expected the demo session to open so the payment
flow could be executed.
68     await expect(page.locator("xpath=/html/body/div[2]/div/
button").nth(0)).to_be_visible(timeout=15000), "Expected the
demo session to open so the payment flow could be executed."
69
70     # --> Could not verify that no duplicate payment confirmation
is shown because the payment/confirmation UI was not reachable
due to the expired QR error.
71     await page.locator("xpath=/html/body/div[2]/div/button").nth
(0).scroll_into_view_if_needed()
72     # Assert-outcome: failed
73     # Assert: Expected to reach the payment confirmation UI to
verify no duplicate confirmation.
74     await expect(page.locator("xpath=/html/body/div[2]/div/
button").nth(0)).to_be_visible(timeout=15000), "Expected to
reach the payment confirmation UI to verify no duplicate
confirmation."
75
76     # --> Test blocked by environment/access constraints during
agent run
77     # Reason: TEST BLOCKED The customer demo session could not be
reached because the QR code presented by the demo is expired
and the UI remains in an access-error state. Observations: -
The page displays 'Erişim Hatası' with the message that the QR
code is no longer valid. - The only actionable control is the
'Tekrar Dene' button; clicking it (two attempts) did not open

```

```

78         a customer/table session or reveal a...
       raise AssertionError("Test blocked during agent run: " + "TEST
       BLOCKED The customer demo session could not be reached because
       the QR code presented by the demo is expired and the UI
       remains in an access-error state. Observations: - The page
       displays 'Eri\u015fim Hatas\u0131' with the message that the
       QR code is no longer valid. - The only actionable control is
       the 'Tekrar Dene' button; clicking it (two attempts) did not
       open a customer/table session or reveal a..." + " - the
       exported script cannot reproduce a PASS in this environment.")
79     await asyncio.sleep(5)
80
81     finally:
82         if context:
83             await context.close()
84         if browser:
85             await browser.close()
86         if pw:
87             await pw.stop()
88
89     asyncio.run(run_test())
90

```

Error

TEST BLOCKED The customer demo session could not be reached because the QR code presented by the demo is expired and the UI remains in an access-error state. Observations: - The page displays 'Erişim Hatası' with the message that the QR code is no longer valid. - The only actionable control is the 'Tekrar Dene' button; clicking it (two attempts) did not open a customer/table session or reveal any order/payment UI. - No order, checkout, or payment elements are accessible on the page, so the payment flow cannot be executed or verified.

Cause

The hosting side is likely serving an expired or invalid QR/session token, or the backend that validates/generates the demo session is misconfigured or unavailable. This can happen if the QR code is hardcoded or cached after expiration, the server-side session TTL is too short, the clock on the host is skewed, or the retry endpoint is failing to create a new valid customer/table session, leaving the app permanently in the access-error state.

Fix

Ensure the demo QR/session token is generated fresh on each deployment/request and that the hosting environment is not serving a stale or expired QR code from cache/build artifacts. Verify backend session initialization on Render is working (environment variables, clock/timezone, persistent store/TTL settings, and any token-expiration logic), and make the "Tekrar Dene" action request a new valid session/QR rather than reusing an expired one. If the app depends on a backend service, confirm it is reachable and returning a valid table/customer session so the UI can transition out of the access-error state.

Place a complete customer order

ATTRIBUTES	
Status	BLOCKED
Priority	High
Description	Validates that a customer can build an order from the QR menu and submit it successfully from the order flow.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080838-fa24546e.webm


```
1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Demoyu Deneyin' button to open the demo role
52         # selector.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section/div[3]/div[2]/
55         # button","desc":"Click 'Demoyu Deneyin button'", "input":null,
56         # "field":"9"}
57         # Demoyu Deneyin button
58         elem = page.get_by_text('Restoran ve Kafeler için')
```

```

Geliştirildi', exact=True).locator("xpath=ancestor-or-self::*
[./button][1]").get_by_role('button', name='Demoyu Deneyin',
exact=True)
48 await elem.click(timeout=10000)
49
50 # -> Click the 'Müşteri Demo' option in the demo selector to
open the customer QR menu experience.
51 # @ts-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/div/div[2]/a",
"desc":"Click 'M\u00fc\u015fteri Demo QR men\u00fcs\u00fc
g\u00f6r\u00fcnt\u00fcleyin, \u00f6r\u00fcnleri... link'",
"input":null,"field":"342"}
52 # Müşteri Demo QR menüyü görüntüleyin, ürünleri... link
53 elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin.', exact=True)
54 await elem.click(timeout=10000)
55
56 # -> Click the visible 'Tekrar Dene' button to retry loading
the customer menu.
57 # @ts-step {"i":4,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"377"}
58 # Tekrar Dene button
59 elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
60 await elem.click(timeout=10000)
61
62 # -> Click the 'Tekrar Dene' button to retry loading the
customer menu.
63 # @ts-step {"i":5,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"387"}
64 # Tekrar Dene button
65 elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
66 await elem.click(timeout=10000)
67
68 # -> Click the 'Tekrar Dene' button to retry loading the
customer menu and check whether the menu loads.
69 # @ts-step {"i":6,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"444"}
70 # Tekrar Dene button
71 elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
72 await elem.click(timeout=10000)
73
74 # --> Assertions to verify final state
75
76 # --> Could not verify a successful order confirmation because
the demo menu is inaccessible due to an access error on the
page.
77 # Assert-outcome: failed
78 # Assert: Expected the page to not display the access error
'Erişim Hatası'.
79 await expect(page.locator("xpath=/html/body/section").nth(0)).
to_contain_text("Eri\u015fim Hatası\u0131", timeout=15000),
"Expected the page to not display the access error

```

```

80         'Eri\u015fim Hatas\u0131'."
81     # --> Could not verify the submitted order in order history
      because the ordering UI was unreachable and the page shows a
      retry button for an access error.
82     # Assert-outcome: failed
83     # Assert: Expected the retry button 'Tekrar Dene' to not be
      visible so the order flow could proceed.
84     await expect(page.locator("xpath=/html/body/div[2]/div/
      button").nth(0)).to_have_text("Tekrar Dene", timeout=15000),
      "Expected the retry button 'Tekrar Dene' to not be visible so
      the order flow could proceed."
85
86     # --> Test blocked by environment/access constraints during
      agent run
87     # Reason: TEST BLOCKED The customer order flow could not be
      exercised because the demo menu is inaccessible – the page
      shows an access error that did not clear after retries.
      Observations: - The page displays the heading 'Erişim Hatası'
      and the message 'Bu QR kod artık geçerli değil. Lütfen işletme
      personelinden yeni QR kod isteyin.' - The visible button
      'Tekrar Dene' was clicked three times but did not ...
88     raise AssertionError("Test blocked during agent run: " + "TEST
      BLOCKED The customer order flow could not be exercised because
      the demo menu is inaccessible \u2014 the page shows an access
      error that did not clear after retries. Observations: - The
      page displays the heading 'Eri\u015fim Hatas\u0131' and the
      message 'Bu QR kod art\u0131k ge\u00e7erli de\u011fil.
      L\u00fctfen i\u015fletme personelinden yeni QR kod isteyin.' -
      The visible button 'Tekrar Dene' was clicked three times but
      did not ..." + " – the exported script cannot reproduce a PASS
      in this environment.")
89     await asyncio.sleep(5)
90
91     finally:
92         if context:
93             await context.close()
94         if browser:
95             await browser.close()
96         if pw:
97             await pw.stop()
98
99     asyncio.run(run_test())
100

```

Error

TEST BLOCKED The customer order flow could not be exercised because the demo menu is inaccessible — the page shows an access error that did not clear after retries. Observations: - The page displays the heading 'Erişim Hatası' and the message 'Bu QR kod artık geçerli değil. Lütfen işletme personelinden yeni QR kod isteyin.' - The visible button 'Tekrar Dene' was clicked three times but did not change the page state; the access error remains. - No menu categories, items, add-to-cart controls, cart, or checkout UI elements are present on the page, so the order flow cannot be performed. Additional details: - URL: /menu/demo-business-id/1 - Retry attempts: 3 Because the required UI (customer menu and ordering controls) is unreachable, the test is blocked and cannot proceed.

Cause

The hosting side is likely resolving the demo menu URL to an access-denied state because the underlying demo QR/menu identifier is invalid, expired, missing from the deployed data, or misconfigured in production. A routing or backend lookup issue may be causing the app to think the QR code is no longer valid, which prevents the menu UI from rendering and blocks the order flow.

Fix

Check the hosting/deployment configuration that serves `/menu/demo-business-id/1` and ensure the demo QR/menu route is enabled in production. If the page depends on backend data, verify the demo business record, menu slug/ID, and QR validity mapping exist and are not expired/disabled. Also inspect environment variables, database seeding, and any middleware/auth/geo/rate-limit rules that may incorrectly return the 'QR code no longer valid' state for the demo path. If the QR token or business identifier changed during deployment, regenerate/update the demo QR and its backend lookup so the public menu page resolves to the customer menu instead of the access error.

Waiter sees a revoked customer session blocked from access

ATTRIBUTES	
Status	FAILED
Priority	Medium
Description	Validates that a waiter can observe a revoked session state and confirm the blocked access outcome in the panel.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-081147-e36f7eb4.webm

```

1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Garson Demo' link to open the waiter (garson)
52         # demo panel.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a[2]",
55         # "desc":"Click 'Garson Demo Gelen sipari\u015fleri
56         # y\u00f6netin, masa... link'", "input":null, "field":"9"}
57         # Garson Demo Gelen siparişleri yönetin, masa... link
58         elem = page.locator('a[href="/auth/signin?demo=waiter"]')

```

```

48     await elem.click(timeout=10000)
49
50     # -> Click the 'Giriş Yap' button to sign in to the waiter
    (Garson) demo panel.
51     # @@ts-step {"i":3,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/button",
    "desc":"Click 'Giriş Yap button'", "input":null,
    "field":"353"}
52     # Giriş Yap button
53     elem = page.get_by_role('button', name='Giriş Yap', exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Masalar' link to open the tables list.
57     # @@ts-step {"i":4,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/nav/a[4]", "desc":"Click
    'Masalar link'", "input":null, "field":"598"}
58     # Masalar link
59     elem = page.get_by_role('link', name='Masalar', exact=True)
60     await elem.click(timeout=10000)
61
62     # -> Open the 'ALDA' table card to view its session(s) and
    session details.
63     # @@ts-step {"i":5,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/div[2]/div/div[2]/div[2]",
    "desc":"Click 'ALDA Boş'", "input":null,
    "field":"711"}
64     # ALDA Boş
65     elem = page.get_by_text('ALDA Boş', exact=True)
66     await elem.click(timeout=10000)
67
68     # -> Click the 'Bu masa şu anda boş' button in the ALDA modal
    to dismiss or reveal options and then inspect other table
    cards for any revoked sessions.
69     # @@ts-step {"i":6,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/div[2]/div/div[3]/div/div/
    button", "desc":"Click 'Bu masa şu anda boş button'", "input":null,
    "field":"731"}
70     # X button
71     elem = page.get_by_role('button', name='X', exact=True)
72     await elem.click(timeout=10000)
73
74     # -> Open the 'Bahçe 1' table card to check for any revoked
    customer sessions and view session details.
75     # @@ts-step {"i":7,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/div[2]/div/div[2]/div",
    "desc":"Click 'Bahçe 1 Boş'", "input":null,
    "field":"710"}
76     # Bahçe 1 Boş
77     elem = page.get_by_text('Bahçe 1 Boş', exact=True)
78     await elem.click(timeout=10000)
79
80     # -> Click the 'Bu masa şu anda boş' button in the Bahçe 1
    modal to dismiss it so other table cards can be inspected.
81     # @@ts-step {"i":8,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/div[2]/div/div[3]/div/div/
    button", "desc":"Click 'Bu masa şu anda boş button'", "input":null,
    "field":"753"}
82     # X button
83     elem = page.get_by_role('button', name='X', exact=True)

```

```

84     await elem.click(timeout=10000)
85
86     # -> Click the 'Bahçe 2' table card to open its modal and
      inspect for any revoked customer session.
87     # @@ts-step {"i":9,"type":"action","action":"click",
      "selector":"xpath=/html/body/div[2]/div[2]/div/div[2]/div[3]",
      "desc":"Click 'Bah\u00e7e 2 \u26aa Bo\u015f'", "input":null,
      "field":"712"}
88     # Bahçe 2 🍷 Boş
89     elem = page.get_by_text('Bahçe 2 🍷 Boş', exact=True)
90     await elem.click(timeout=10000)
91
92     # -> Close the 'Bahçe 2' modal by clicking the 'Bu masa şu
      anda boş' button so the tables list is visible and then open
      the 'Test 99' table card.
93     # @@ts-step {"i":10,"type":"action","action":"click",
      "selector":"xpath=/html/body/div[2]/div[2]/div/div[3]/div/div/
      button","desc":"Click '\u2715 button'", "input":null,
      "field":"775"}
94     # ✕ button
95     elem = page.get_by_role('button', name='✕', exact=True)
96     await elem.click(timeout=10000)
97
98     # -> Open the 'Test 99' table card from the Masalar (Tables)
      view to inspect its session details.
99     # @@ts-step {"i":11,"type":"action","action":"click",
      "selector":"xpath=/html/body/div[2]/div[2]/div/div[2]/div[4]",
      "desc":"Click 'Test 99 \u26aa Bo\u015f'", "input":null,
      "field":"713"}
100    # Test 99 🍷 Boş
101    elem = page.get_by_text('Test 99 🍷 Boş', exact=True)
102    await elem.click(timeout=10000)
103
104    # --> Assertions to verify final state
105
106    # --> The session was not shown as revoked; the table detail
      modal shows 'Bu masa şu anda boş'.
107    # Assert-outcome: failed
108    # Assert: Expected the session details to show the session as
      revoked.
109    await expect(page.locator("xpath=/html/body/div[2]/div[2]/div/
      div[3]").nth(0)).to_contain_text("Bu masa \u015fu anda
      bo\u015f", timeout=15000), "Expected the session details to
      show the session as revoked."
110
111    # --> Blocked access information was not displayed; the table
      detail modal shows 'Bu masa şu anda boş'.
112    # Assert-outcome: failed
113    # Assert: Expected blocked access information to be displayed
      in the session details.
114    await expect(page.locator("xpath=/html/body/div[2]/div[2]/div/
      div[3]").nth(0)).to_contain_text("Bu masa \u015fu anda
      bo\u015f", timeout=15000), "Expected blocked access
      information to be displayed in the session details."
115    await asyncio.sleep(5)
116
117    finally:
118        if context:
119            await context.close()

```



```
120         if browser:
121             await browser.close()
122         if pw:
123             await pw.stop()
124
125     asyncio.run(run_test())
126
```

Error

TEST FAILURE The revoked-customer-session state and blocked-access information were not present in the waiter panel, so the required verification could not be completed. Observations: - The table detail modals for ALDA, Bahçe 1, Bahçe 2, and Test 99 each show 'Bu masa şu anda boş' (the table is currently empty). - No session entries, 'revoked' label, or any blocked-access message was visible in any opened table modal.

Cause

The hosted service likely has an environment/data mismatch: the production instance is either pointing at an empty or incorrect database, missing the revoked customer-session records, or failing to load the blocked-access status from the backend. As a result, the waiter panel only sees empty-table state ('Bu masa şu anda boş') instead of the expected revoked-session and access-block information. A stale deployment or cache could also be serving older logic that no longer surfaces these records.

Fix

Verify the production data source/state used by the waiter panel and seed or restore the expected customer-session records for the test environment. If session revocation and blocked-access status are computed from backend data, ensure the API/DB is reachable, migrations and seed scripts are current, and the waiter panel is requesting the correct endpoint/tenant/table identifiers. Also clear any stale cached frontend state or service-worker cache so the latest session-status logic and data are rendered. If the app is intended to show revoked sessions for empty tables, adjust the backend/frontend to expose that state even when a table currently has no active order.

Retry access after an invalid QR code error

ATTRIBUTES	
Status	FAILED
Priority	Medium
Description	Validates that a customer can use the retry path after an invalid QR code access error and return toward the menu flow.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080927-24745fe4.webm

```

1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com", "input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Müşteri Demo' link to start the customer demo
52         # flow.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
55         # "desc":"Click 'M\u00fcs\u00fcter\u00fcleri Demo QR men\u00fc\u00fcs\u00fc
56         # g\u00f6r\u00fcm\u00fcs\u00fcn\u00fcleri... link'",
57         # "input":null,"field":"11"}
58         # M\u00fcs\u00fcter\u00fcleri Demo QR men\u00fc\u00fcs\u00fcn\u00fcleri... link

```

```

47     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin. Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Tekrar Dene' button on the "Erişim Hatası"
(Access Error) page to attempt the retry path.
51     # @@ts-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"327"}
52     # Tekrar Dene button
53     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Tekrar Dene' button to retry and verify the
demo flow advances toward the menu or next demo screen.
57     # @@ts-step {"i":4,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"337"}
58     # Tekrar Dene button
59     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
60     await elem.click(timeout=10000)
61
62     # -> Go back to the previous page (return to the demo landing)
so the demo flow can be restarted from the initial 'Müşteri
Demo' link.
63     # @@ts-step {"i":5,"type":"action","action":"navigate",
"selector":null,"desc":"Navigate","input":null,"field":null}
64     await page.go_back()
65
66     # -> Click the 'Müşteri Demo' link labeled 'Müşteri Demo –
Deneyimle' to start the customer demo flow.
67     # @@ts-step {"i":6,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
"desc":"Click 'Müşteri Demo QR menü\fcy\fc
g\fc\fcnt\fcleyin, \fc\fcnleri... link'",
"input":null,"field":"615"}
68     # Müşteri Demo QR menüyü görüntüleyin, ürünleri... link
69     elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
edin. Deneyimle', exact=True)
70     await elem.click(timeout=10000)
71
72     # -> Click the 'Tekrar Dene' button on the 'Erişim Hatası'
(Access Error) page to attempt the retry path.
73     # @@ts-step {"i":7,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div/button","desc":"Click
'Tekrar Dene button',"input":null,"field":"724"}
74     # Tekrar Dene button
75     elem = page.get_by_role('button', name='Tekrar Dene',
exact=True)
76     await elem.click(timeout=10000)
77
78     # --> Assertions to verify final state
79
80     # --> Clicking the retry action did not advance the demo; the
access error page remained instead of moving to the next

```

```

access step.
81      # Assert-outcome: failed
82      # Assert: Expected the app to navigate away from the access
      error URL after clicking retry.
83      await expect(page).to_have_url(re.compile("menu/
      demo\\-business\\-id/1"), timeout=15000), "Expected the app to
      navigate away from the access error URL after clicking retry."
84      await page.locator("xpath=/html/body/div[2]/div/button").nth
      (0).scroll_into_view_if_needed()
85      # Assert-outcome: failed
86      # Assert: Expected the access error 'Tekrar Dene' button to
      disappear after retry.
87      await expect(page.locator("xpath=/html/body/div[2]/div/
      button").nth(0)).to_be_visible(timeout=15000), "Expected the
      access error 'Tekrar Dene' button to disappear after retry."
88      await asyncio.sleep(5)
89
90      finally:
91          if context:
92              await context.close()
93          if browser:
94              await browser.close()
95          if pw:
96              await pw.stop()
97
98      asyncio.run(run_test())
99

```

Error

TEST FAILURE Clicking the retry action did not advance the demo flow — the access error persisted and no navigation toward the menu was observed. Observations: - The page displays 'Erişim Hatası' with the message 'Bu QR kod artık geçerli değil. Lütfen işletme personelininden yeni QR kod isteyin.' and a visible 'Tekrar Dene' button. - The 'Tekrar Dene' button was clicked multiple times during this session, but the page remained on the same error screen (URL: <https://qr-menu-product.onrender.com/menu/demo-business-id/1>). - No transition, menu, or next access step was reached after retry attempts.

Cause

The hosting-side flow appears to be serving a permanently invalidated demo access state: the menu route is resolving to the access-error screen and the retry action is not leading to a new server response or route transition. This typically happens when the demo QR/business ID is expired or missing in production data, when the retry button is wired but does not call a refresh/navigation function, or when cached SSR/static content continues returning the same error page for every retry.

Fix

Make the demo retry path actually refresh/re-resolve the access state and route the user to a valid menu state. Concretely: ensure the Tekrar Dene button triggers the correct client-side handler (not a no-op), verify the demo business/menu record exists and is marked valid on the server, and update any access/expiry logic so the demo QR code is not permanently rejected. If the page is statically cached or server-rendered with an expired token, invalidate the cache and have retry re-fetch from the backend and/or redirect to a fresh demo URL instead of staying on </menu/demo-business-id/1>.

Review revenue summary

ATTRIBUTES	
Status	BLOCKED
Priority	High
Description	Verify an admin can open the revenue view and see revenue metrics for operational oversight.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080955-6fa164c4.webm

```
1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Admin Demo' link to open the admin dashboard.
52         # @@ts-step {"i":2,"type":"action","action":"click",
53         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a[3]",
54         # "desc":"Click 'Admin Demo' link", "input":null, "field":"14"}
55         # Admin Demo link, ürün, masa, kategori ve personel... link
56         elem = page.get_by_role('link', name='Admin Demo link, ürün,
57         masa, kategori ve personel yönetimini deneyimleyin.',
```

```

Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Giriş Yap' button to sign in to the admin
    dashboard.
51     # @ts-step {"i":3,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/button",
    "desc":"Click 'Giriş Yap button'", "input":null,
    "field":"356"}
52     # Giriş Yap button
53     elem = page.get_by_role('button', name='Giriş Yap', exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Geçmiş Ödemeler' link in the left navigation
    to open the past payments / revenue view.
57     # @ts-step {"i":4,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/aside/nav/div[2]/a[4]",
    "desc":"Click 'Geçmiş Ödemeler link'",
    "input":null, "field":"523"}
58     # Geçmiş Ödemeler link
59     elem = page.get_by_role('link', name='Geçmiş Ödemeler',
    exact=True)
60     await elem.click(timeout=10000)
61
62     # -> Click the 'Tümü' button in the payments filters to select
    the all-period reporting view.
63     # @ts-step {"i":5,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/
    button", "desc":"Click 'Tümü button'", "input":null,
    "field":"936"}
64     # Tümü button
65     elem = page.get_by_role('button', name='Tümü', exact=True)
66     await elem.click(timeout=10000)
67
68     # --> Assertions to verify final state
69
70     # --> The Payments page shows revenue-related UI: the payments
    table header is visible.
71     # Assert-outcome: failed
72     # Assert: Expected the payments table header to be visible.
73     await expect(page.locator("xpath=/html/body/div[2]/main/div[2]/
    div/div[3]/table/thead/tr").nth(0)).to_contain_text("Masa",
    timeout=15000), "Expected the payments table header to be
    visible."
74
75     # --> A reporting-period / date-range selector (e.g., labeled
    'Tarih' or 'Tarih Aralığı') is not present, so revenue for a
    selected reporting period cannot be verified.
76     # Assert-outcome: failed
77     # Assert: Expected the Payments page to contain a date-range
    selector labeled 'Tarih'.
78     await expect(page.locator("xpath=/html/body/div[2]/main/div[2]/
    div/div[3]/table/thead/tr").nth(0)).to_contain_text("Tarih",
    timeout=15000), "Expected the Payments page to contain a
    date-range selector labeled 'Tarih'."
79
80     # --> Test blocked by environment/access constraints during
    agent run
81     # Reason: TEST BLOCKED A reporting-period/date-range selector

```



```

(e.g., 'Tarih Aralığı' or a calendar picker) is not present on
the Payments page, so the requested period-selection step
cannot be performed. Observations: - The Payments page
displays revenue metrics ('Bugün Alınan Ciro') and an
interactive payments table with dated rows (examples: 25.08.
2026, 24.08.2026). - No date-range control or calendar ...
82     raise AssertionError("Test blocked during agent run: " + "TEST
BLOCKED A reporting-period/date-range selector (e.g., 'Tarih
Aralı\u0131\u011f\u0131' or a calendar picker) is not present
on the Payments page, so the requested period-selection step
cannot be performed. Observations: - The Payments page
displays revenue metrics ('Bug\u00fcn Al\u0131nan Ciro') and
an interactive payments table with dated rows (examples: 25.08.
2026, 24.08.2026). - No date-range control or calendar ..." +
" - the exported script cannot reproduce a PASS in this
environment.")
83     await asyncio.sleep(5)
84
85     finally:
86         if context:
87             await context.close()
88         if browser:
89             await browser.close()
90         if pw:
91             await pw.stop()
92
93     asyncio.run(run_test())
94

```

Error

TEST BLOCKED A reporting-period/date-range selector (e.g., 'Tarih Aralığı' or a calendar picker) is not present on the Payments page, so the requested period-selection step cannot be performed. Observations: - The Payments page displays revenue metrics ('Bugün Alınan Ciro') and an interactive payments table with dated rows (examples: 25.08.2026, 24.08.2026). - No date-range control or calendar picker labeled 'Tarih'/'Tarih Aralığı' was found on the page. - Filter buttons ('Tümü', 'Onay Bekleyenler', 'Tamamlananlar', 'Reddedilenler') are present but do not provide arbitrary date-range selection.

Cause

The hosting-side issue is most likely that the deployed Payments page is missing the date-range filter control in production, either because the feature was never included in the build, is disabled by configuration/feature flag, or is hidden due to a UI/CSS/regression issue on the hosted deployment. Since only preset status filters are visible, the test cannot perform the required arbitrary period selection step.

Fix

Add a visible reporting-period/date-range selector on the Payments page (e.g., a 'Tarih Aralığı' date picker or start/end date inputs) and wire it to filter the payments table and revenue metrics. If the control already exists in code, check the deployed build/config to ensure the component is rendered in production, not hidden by a feature flag, responsive breakpoint, permissions issue, or CSS overflow/z-index problem. Also verify the page is not serving an outdated bundle/cached version missing this UI, and redeploy the corrected frontend so the selector is accessible to automated tests.

Add a new staff member

ATTRIBUTES	
Status	BLOCKED
Priority	High
Description	Verify an admin can create a staff account and see it listed in personnel management.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-080944-d9e5b24e.webm

```

1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Admin Demo' link on the homepage to open the
52         # admin demo flow.
53         # @@ts-step {"i":2,"type":"action","action":"click",
54         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a[3]",
55         # "desc":"Click 'Admin Demo' link", "input":null, "field":"10"}
56         # Admin Demo link, ürün, masa, kategori ve personel... link
57         elem = page.get_by_role('link', name='Admin Demo link, ürün,

```

```

masa, kategori ve personel yönetimini deneyimleyin.
Deneyimle', exact=True)
48 await elem.click(timeout=10000)
49
50 # -> Click the 'Admin · admin@demo.com' demo button to select
the admin demo account.
51 # @ts-step {"i":3,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[3]/div[3]/div/button",
"desc":"Click 'Admin \u00b7 admin@demo.com button'",
"input":null,"field":"370"}
52 # Admin · admin@demo.com button
53 elem = page.get_by_role('button', name='Admin · admin@demo.
com', exact=True)
54 await elem.click(timeout=10000)
55
56 # -> Click the 'Admin · admin@demo.com' demo button to select
the admin demo account.
57 # @ts-step {"i":4,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/button",
"desc":"Click 'Giri\u015f Yap button'", "input":null,
"field":"355"}
58 # Giriş Yap button
59 elem = page.get_by_role('button', name='Giriş Yap', exact=True)
60 await elem.click(timeout=10000)
61
62 # -> Click the 'Personel' link in the left menu to open
Personnel (staff) management.
63 # @ts-step {"i":5,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/aside/nav/div[4]/a[2]",
"desc":"Click 'Personel link'", "input":null, "field":"553"}
64 # Personel link
65 elem = page.get_by_text('İşletme', exact=True).locator
("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link',
name='Personel', exact=True)
66 await elem.click(timeout=10000)
67
68 # -> Fill 'TESTSTAFF001' into the 'Özel kod (boş bırakılırsa
otomatik oluşturulur)' field and click the 'Oluştur' button.
69 # @ts-step {"i":6,"type":"action","action":"fill",
"selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/div/
input","desc":"Fill 'TESTSTAFF001' into \u00d6zel kod
(bo\u015f b\u0131rakılırsa otomatik
olu\u015fturulur) text field","input":"TESTSTAFF001",
"field":"925"}
70 # Özel kod (boş bırakılırsa otomatik oluşturulur) text field
71 elem = page.get_by_placeholder('Özel kod (boş bırakılırsa
otomatik oluşturulur)', exact=True)
72 await elem.wait_for(state="visible", timeout=10000)
73 await elem.fill("TESTSTAFF001")
74
75 # -> Fill 'TESTSTAFF001' into the 'Özel kod (boş bırakılırsa
otomatik oluşturulur)' field and click the 'Oluştur' button.
76 # @ts-step {"i":7,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/div/
button","desc":"Click 'Olu\u015ftur button'", "input":null,
"field":"926"}
77 # Oluştur button
78 elem = page.get_by_role('button', name='Oluştur', exact=True)
79 await elem.click(timeout=10000)

```

```

80
81     # --> Assertions to verify final state
82
83     # --> The newly created staff member is not shown in the
    personnel list.
84     # Assert-outcome: failed
85     # Assert: Expected staff list to show the newly created staff
    'TESTSTAFF001' in the name column.
86     await expect(page.locator("xpath=/html/body/div[2]/main/div[2]/
    div/div[1]/div[2]/table/tbody/tr/td[1]").nth(0)).
    to_contain_text("TESTSTAFF001", timeout=15000), "Expected
    staff list to show the newly created staff 'TESTSTAFF001' in
    the name column."
87
88     # --> Test blocked by environment/access constraints during
    agent run
89     # Reason: TEST BLOCKED The test could not be run – the admin
    UI does not provide fields to create a staff account directly
    (name/email/role). Staff creation appears to rely on invite
    codes that staff use to register themselves, so the scripted
    steps to fill name/email/role and save cannot be executed from
    the admin panel. Observations: - The Personnel List shows only
    'Demo Garson' (no new staff row appea...
90     raise AssertionError("Test blocked during agent run: " + "TEST
    BLOCKED The test could not be run \u2014 the admin UI does not
    provide fields to create a staff account directly (name/email/
    role). Staff creation appears to rely on invite codes that
    staff use to register themselves, so the scripted steps to
    fill name/email/role and save cannot be executed from the
    admin panel. Observations: - The Personnel List shows only
    'Demo Garson' (no new staff row appea..." + " – the exported
    script cannot reproduce a PASS in this environment.")
91     await asyncio.sleep(5)
92
93     finally:
94         if context:
95             await context.close()
96         if browser:
97             await browser.close()
98         if pw:
99             await pw.stop()
100
101     asyncio.run(run_test())
102

```

Error

TEST BLOCKED The test could not be run — the admin UI does not provide fields to create a staff account directly (name/email/role). Staff creation appears to rely on invite codes that staff use to register themselves, so the scripted steps to fill name/email/role and save cannot be executed from the admin panel. Observations: - The Personnel List shows only 'Demo Garson' (no new staff row appeared). - The admin UI exposes only an invite-code input labeled 'Özel kod (boş bırakılırsa otomatik oluşturulur)' and an 'Oluştur' button; creating the code 'TESTSTAFF001' produced repeated 'Bu davet kodu zaten kullanılıyor' alerts while the code appears in the 'Davet Kodları' list.

Cause

The hosted service likely implements staff onboarding only through self-registration via invite codes, not through admin-side account creation. As a result, the admin UI lacks the expected inputs the test tries to use, and the invite-code creation path may also reject reused/generated codes or be wired to a separate table without creating a visible personnel record. This mismatch between the expected admin workflow and the actual hosted implementation blocks the test.

Fix

Add a direct staff-management form in the admin panel with fields for name, email, and role, and make it create the staff record immediately (or add a separate admin action to convert an invite code into a staff account). Ensure the invite-code flow does not block admin testing, and return a clear success state after creation. Also verify that the backend correctly persists new staff rows and that the personnel list refreshes from the created record.

Create a product category

ATTRIBUTES	
Status	FAILED
Priority	High
Description	Verify an admin can add a new category and see it available for organizing products.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-081346-df33f825.webm

```
1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Click the 'Admin Demo' link to open the admin demo flow.
52         # @@ts-step {"i":2,"type":"action","action":"click",
53         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a[3]",
54         # "desc":"Click 'Admin Demo Ciro, \u00fcr\u00fc\u00fcr\u00fc, masa, kategori
55         # ve personel... link'", "input":null, "field":"13"}
56         # Admin Demo Ciro, ürün, masa, kategori ve personel... link
57         elem = page.get_by_role('link', name='Admin Demo Ciro, ürün,
58         masa, kategori ve personel yönetimini deneyimleyin,
```



```

Deneyimle', exact=True)
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Giriş Yap' button to sign in to the admin
    dashboard.
51     # @ts-step {"i":3,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/button",
    "desc":"Click 'Giriş Yap button'", "input":null,
    "field":"355"}
52     # Giriş Yap button
53     elem = page.get_by_role('button', name='Giriş Yap', exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Click the 'Kategoriler' link in the left menu to open the
    Categories management page.
57     # @ts-step {"i":4,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/aside/nav/div[3]/a[2]",
    "desc":"Click 'Kategoriler link'", "input":null, "field":"536"}
58     # Kategoriler link
59     elem = page.get_by_role('link', name='Kategoriler', exact=True)
60     await elem.click(timeout=10000)
61
62     # -> Click the '+ Yeni Kategori' button to open the
    create-category form.
63     # @ts-step {"i":5,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/main/div[2]/div/div/
    button", "desc":"Click '+ Yeni Kategori button'", "input":null,
    "field":"920"}
64     # + Yeni Kategori button
65     elem = page.get_by_role('button', name='+ Yeni Kategori',
    exact=True)
66     await elem.click(timeout=10000)
67
68     # -> Fill 'E2E Test Kategori' into the 'Kategori Adı' field,
    set the icon to '👉' in the 'İkon (Emoji)' field, then click
    the 'Oluştur' button.
69     # @ts-step {"i":6,"type":"action","action":"fill",
    "selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/div/
    div/div/input", "desc":"Fill 'E2E Test Kategori' into \u00d6rn:
    S\u00131cak \u000e7ecekler text field", "input":"E2E Test
    Kategori", "field":"1003"}
70     # Örn: Sıcak İçecekler text field
71     elem = page.get_by_placeholder('Örn: Sıcak İçecekler',
    exact=True)
72     await elem.wait_for(state="visible", timeout=10000)
73     await elem.fill("E2E Test Kategori")
74
75     # -> Fill 'E2E Test Kategori' into the 'Kategori Adı' field,
    set the icon to '👉' in the 'İkon (Emoji)' field, then click
    the 'Oluştur' button.
76     # @ts-step {"i":7,"type":"action","action":"fill",
    "selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/div/
    div/div[2]/input", "desc":"Fill '\ud83c\udff7\ufe0f' into
    \u00d6rn: \u2615 text field", "input":"\ud83c\udff7\ufe0f",
    "field":"1005"}
77     # Örn: 🍷 text field
78     elem = page.get_by_placeholder('Örn: 🍷', exact=True)
79     await elem.wait_for(state="visible", timeout=10000)
80     await elem.fill("\ud83c\udff7\ufe0f")

```

```
81
82     # -> Fill 'E2E Test Kategori' into the 'Kategori Adı' field,
      set the icon to '👉' in the 'İkon (Emoji)' field, then click
      the 'Oluştur' button.
83     # @@ts-step {"i":8,"type":"action","action":"click",
      "selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/div/
      div/div[4]/button","desc":"Click 'Olu\u015ftur button'",
      "input":null,"field":"1012"}
84     # Oluştur button
85     elem = page.get_by_role('button', name='Oluştur', exact=True)
86     await elem.click(timeout=10000)
87
88     # -> Click the 'Oluştur' button in the 'Yeni Kategori' modal
      to submit and create the new category.
89     # @@ts-step {"i":9,"type":"action","action":"click",
      "selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/div/
      div/div[4]/button","desc":"Click 'Olu\u015ftur button'",
      "input":null,"field":"1012"}
90     # Oluştur button
91     elem = page.get_by_role('button', name='Oluştur', exact=True)
92     await elem.click(timeout=10000)
93
94     # -> Click the 'Oluştur' button to submit the new category
95     # @@ts-step {"i":10,"type":"action","action":"click",
      "selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/div/
      div/div[4]/button","desc":"Click 'Olu\u015ftur button'",
      "input":null,"field":"1012"}
96     # Oluştur button
97     elem = page.get_by_role('button', name='Oluştur', exact=True)
98     await elem.click(timeout=10000)
99
100    # -> Click the 'Oluştur' button to submit the new category.
101    # @@ts-step {"i":11,"type":"action","action":"click",
      "selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/div/
      div/div[4]/button","desc":"Click 'Olu\u015ftur button'",
      "input":null,"field":"1012"}
102    # Oluştur button
103    elem = page.get_by_role('button', name='Oluştur', exact=True)
104    await elem.click(timeout=10000)
105
106    # -> Click the 'İptal' button to close the 'Yeni Kategori'
      modal and check the categories list for 'E2E Test Kategori'.
107    # @@ts-step {"i":12,"type":"action","action":"click",
      "selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/div/
      div/div[4]/button[2]","desc":"Click '\u0130ptal button'",
      "input":null,"field":"1013"}
108    # İptal button
109    elem = page.get_by_role('button', name='İptal', exact=True)
110    await elem.click(timeout=10000)
111
112    # -> Click the '+ Yeni Kategori' button to open the
      create-category form and inspect the visible fields and any
      validation or error messages.
113    # @@ts-step {"i":13,"type":"action","action":"click",
      "selector":"xpath=/html/body/div[2]/main/div[2]/div/div/
      button","desc":"Click '+ Yeni Kategori button'", "input":null,
      "field":"920"}
114    # + Yeni Kategori button
115    elem = page.get_by_role('button', name='+ Yeni Kategori',
```

```

exact=True)
116     await elem.click(timeout=10000)
117
118     # -> Fill 'E2E Test Kategori' into the 'Kategori Adı' field,
    fill '🍷' into the 'İkon (Emoji)' field, then click the
    'Oluştur' button to submit.
119     # @@ts-step {"i":14,"type":"action","action":"fill",
    "selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/div/
    div/div/input","desc":"Fill 'E2E Test Kategori' into \u00d6rn:
    S\u00131cak \u0013\u0007ecekler text field","input":"E2E Test
    Kategori","field":"1065"}
120     # Örn: Sıcak İçecekler text field
121     elem = page.get_by_placeholder('Örn: Sıcak İçecekler',
    exact=True)
122     await elem.wait_for(state="visible", timeout=10000)
123     await elem.fill("E2E Test Kategori")
124
125     # -> Fill 'E2E Test Kategori' into the 'Kategori Adı' field,
    fill '🍷' into the 'İkon (Emoji)' field, then click the
    'Oluştur' button to submit.
126     # @@ts-step {"i":15,"type":"action","action":"fill",
    "selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/div/
    div/div[2]/input","desc":"Fill '\ud83c\uddff7\u2013' into
    \u00d6rn: \u2615 text field","input":"\ud83c\uddff7\u2013",
    "field":"1067"}
127     # Örn: 🍷 text field
128     elem = page.get_by_placeholder('Örn: 🍷', exact=True)
129     await elem.wait_for(state="visible", timeout=10000)
130     await elem.fill("\ud83c\uddff7\u2013")
131
132     # -> Fill 'E2E Test Kategori' into the 'Kategori Adı' field,
    fill '🍷' into the 'İkon (Emoji)' field, then click the
    'Oluştur' button to submit.
133     # @@ts-step {"i":16,"type":"action","action":"click",
    "selector":"xpath=/html/body/div[2]/main/div[2]/div/div[2]/div/
    div/div[4]/button","desc":"Click 'Olu\u015ftur button'",
    "input":null,"field":"1074"}
134     # Oluştur button
135     elem = page.get_by_role('button', name='Oluştur', exact=True)
136     await elem.click(timeout=10000)
137
138     # --> Assertions to verify final state
139
140     # --> New category 'E2E Test Kategori' was not added to the
    category list because the create dialog remained open.
141     # Assert-outcome: failed
142     # Assert: Expected the 'Yeni Kategori' modal to be closed
    after creating the category.
143     await expect(page.locator("xpath=/html/body/div[2]/main/div[2]/
    div/div[2]")).nth(0).not_to_be_visible(timeout=15000),
    "Expected the 'Yeni Kategori' modal to be closed after
    creating the category."
144     await asyncio.sleep(5)
145
146     finally:
147         if context:
148             await context.close()
149         if browser:
150             await browser.close()

```

```
151         if pw:
152             await pw.stop()
153
154     asyncio.run(run_test())
155
```

Error

TEST FAILURE Creating the new category did not complete — the create action did not add 'E2E Test Kategori' to the category list. Observations: - The 'Yeni Kategori' modal remained open after multiple clicks on the 'Oluştur' button and the dialog was not dismissed. - A search of the categories list returned no match for 'E2E Test Kategori'. - No visible validation or error message appeared on the page or inside the modal to explain the failure.

Cause

The most likely cause is that the 'Oluştur' action is reaching a broken or inaccessible backend service in production, so the category is never persisted and the modal never receives a success response to dismiss itself. This can happen if the host is missing required env vars, the API route is returning an error, or the database write is failing (permission/connection/schema issue).

Fix

Check the category-create request in production logs and browser network traffic. A likely hosting-side issue is that the form submit/API call is failing on the server (for example: missing environment variables for the database, a broken backend route, a 500/403 response, or a write operation blocked by the hosted database connection). Fix by restoring the backend create-category endpoint, ensuring the production database connection and credentials are configured correctly on the host, and returning/handling errors in the UI so the modal closes only on success and shows a validation/error message on failure.

Customer sees payment status update after completion

ATTRIBUTES	
Status	BLOCKED
Priority	Medium
Description	Verify the customer payment status updates from pending to completed within the active session after payment is finalized.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260826-081557-5ce45586.webm

```

1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         # @@ts-step {"i":1,"type":"action","action":"navigate",
41         # "selector":null,"desc":"Navigate to https://qr-menu-product.
42         # onrender.com","input":"https://qr-menu-product.onrender.com",
43         # "field":null}
44         await page.goto("https://qr-menu-product.onrender.com")
45         try:
46             await page.wait_for_load_state("domcontentloaded",
47                 timeout=5000)
48         except Exception:
49             pass
50
51         # -> Open the 'Garson Demo' demo by clicking the 'Deneyimle' /
52         # 'Garson Demo' link to access the waiter panel and locate
53         # active sessions with pending payments.
54         # @@ts-step {"i":2,"type":"action","action":"click",
55         # "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a[2]",
56         # "desc":"Click 'Garson Demo Gelen sipari\u015fleri
57         # y\u00f6netin, masa... link'", "input":null, "field":"13"}
58         # Garson Demo Gelen siparisleri yönetin. masa... link

```

```

47     elem = page.locator('a[href="/auth/signin?demo=waiter"]')
48     await elem.click(timeout=10000)
49
50     # -> Click the 'Giriş Yap' button to sign in to the Garson
        (waiter) panel.
51     # @ts-step {"i":3,"type":"action","action":"click",
        "selector":"xpath=/html/body/div[2]/div[3]/div[2]/form/button",
        "desc":"Click 'Giriş Yap button'", "input":null,
        "field":"353"}
52     # Giriş Yap button
53     elem = page.get_by_role('button', name='Giriş Yap', exact=True)
54     await elem.click(timeout=10000)
55
56     # -> Open the 'Ödemeler' (Payments) view by clicking the
        'Ödemeler' link to look for active sessions with pending
        payments.
57     # @ts-step {"i":4,"type":"action","action":"click",
        "selector":"xpath=/html/body/div[2]/nav/a[3]", "desc":"Click
        '\u00d6demeler link'", "input":null, "field":"514"}
58     # Ödemeler link
59     elem = page.get_by_role('link', name='Ödemeler', exact=True)
60     await elem.click(timeout=10000)
61
62     # -> Click the 'Masalar' (Tables) link to open the tables view
        and look for an active table session with a pending payment.
63     # @ts-step {"i":5,"type":"action","action":"click",
        "selector":"xpath=/html/body/div[2]/nav/a[4]", "desc":"Click
        'Masalar link'", "input":null, "field":"515"}
64     # Masalar link
65     elem = page.get_by_role('link', name='Masalar', exact=True)
66     await elem.click(timeout=10000)
67
68     # -> Open the table card labeled 'ALDA' to inspect its session
        and payment status.
69     # @ts-step {"i":6,"type":"action","action":"click",
        "selector":"xpath=/html/body/div[2]/div[2]/div/div[2]/div[2]",
        "desc":"Click 'ALDA \u26aa Boş'", "input":null,
        "field":"757"}
70     # ALDA 🟡 Boş
71     elem = page.get_by_text('ALDA 🟡 Boş', exact=True)
72     await elem.click(timeout=10000)
73
74     # -> Open the 'Ödemeler' (Payments) view to look for an active
        session with a pending payment.
75     # @ts-step {"i":7,"type":"action","action":"navigate",
        "selector":null, "desc":"Navigate to https://qr-menu-product.
        onrender.com/waiter/payments", "input": "https://qr-menu-product.
        onrender.com/waiter/payments", "field":null}
76     await page.goto("https://qr-menu-product.onrender.com/waiter/
        payments")
77     try:
78         await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
79     except Exception:
80         pass
81
82     # -> Click the 'Masalar' (Tables) tab to locate an active
        customer session that might have a pending payment.
83     # @ts-step {"i":8,"type":"action","action":"click",

```

```

84     "selector":"xpath=/html/body/div[2]/nav/a[4]","desc":"Click
      'Masalar link'","input":null,"field":"804"}
85   # Masalar link
86   elem = page.get_by_role('link', name='Masalar', exact=True)
87   await elem.click(timeout=10000)
88
89   # -> Open the application homepage and click the 'Demo Kafe'
      (Customer demo) link to create a customer session that can
      request payment.
90   # @@@@-step {"i":9,"type":"action","action":"navigate",
      "selector":null,"desc":"Navigate to https://qr-menu-product.
      onrender.com","input":"https://qr-menu-product.onrender.com",
      "field":null}
91   # Open URL in new tab
92   page = await context.new_page()
93   await page.goto("https://qr-menu-product.onrender.com")
94   try:
95       await page.wait_for_load_state("domcontentloaded",
          timeout=5000)
96   except Exception:
97       pass
98
99   # -> Click the 'Müşteri Demo' (Customer Demo) link labeled
      'Deneyimle' to open the customer ordering view.
100  # @@@@-step {"i":10,"type":"action","action":"click",
      "selector":"xpath=/html/body/div[2]/section[4]/div[2]/a",
      "desc":"Click 'M\u00fcs\u00fcferi Demo QR men\u00fcy\u00fc
      g\u00fcs\u00fcnt\u00fcleyin, \u00fcs\u00fcnleri... link'",
      "input":null,"field":"47"}
101  # Müşteri Demo QR menüyü görüntüleyin, ürünleri... link
102  elem = page.get_by_role('link', name='Müşteri Demo QR menüyü
      görüntüleyin, ürünleri sepete ekleyin ve sipariş akışını test
      edin. Deneyimle', exact=True)
103  await elem.click(timeout=10000)
104
105  # -> Click the 'Tekrar Dene' (Retry) button on the customer
      demo page to attempt reopening the demo session.
106  # @@@@-step {"i":11,"type":"action","action":"click",
      "selector":"xpath=/html/body/div[2]/div/button","desc":"Click
      'Tekrar Dene button',"input":null,"field":"324"}
107  # Tekrar Dene button
108  elem = page.get_by_role('button', name='Tekrar Dene',
      exact=True)
109  await elem.click(timeout=10000)
110
111  # -> Click the 'Tekrar Dene' button on the customer demo page
      to retry opening the customer ordering session.
112  # @@@@-step {"i":12,"type":"action","action":"click",
      "selector":"xpath=/html/body/div[2]/div/button","desc":"Click
      'Tekrar Dene button',"input":null,"field":"334"}
113  # Tekrar Dene button
114  elem = page.get_by_role('button', name='Tekrar Dene',
      exact=True)
115  await elem.click(timeout=10000)
116
117  # -> Switch to the waiter tab showing the 'Masalar' and
      'Ödemeler' views so the table controls can be inspected to
      generate a fresh QR or start a customer session.

```



```

"selector":null,"desc":"Navigate to BC1A","input":"BC1A",
"field":null}
118 # Switch to tab BC1A
119 page = context.pages[-1] # switch to most recently active tab
120
121 # -> Open the 'ALDA' table card and inspect the modal for
controls to start a session or generate a QR.
122 # @@ts-step {"i":14,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[2]/div/div[2]/div[2]",
"desc":"Click 'ALDA \u26aa Bo\u015f'", "input":null,
"field":"1137"}
123 # ALDA ● Boş
124 elem = page.get_by_text('ALDA ● Boş', exact=True)
125 await elem.click(timeout=10000)
126
127 # -> Click the 'Bu masa şu anda boş' button in the ALDA table
modal to try to open/start a customer session or reveal QR/
session controls.
128 # @@ts-step {"i":15,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[2]/div/div[3]/div/div/
button","desc":"Click '\u2715 button'", "input":null,
"field":"1158"}
129 # ✕ button
130 elem = page.get_by_role('button', name='✕', exact=True)
131 await elem.click(timeout=10000)
132
133 # -> Open the 'ALDA' table card to reveal controls to start a
session or generate a QR.
134 # @@ts-step {"i":16,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[2]/div/div[2]/div[2]",
"desc":"Click 'ALDA \u26aa Bo\u015f'", "input":null,
"field":"1137"}
135 # ALDA ● Boş
136 elem = page.get_by_text('ALDA ● Boş', exact=True)
137 await elem.click(timeout=10000)
138
139 # -> Click the 'Bu masa şu anda boş' button in the ALDA modal
to start a table session or generate a QR.
140 # @@ts-step {"i":17,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[2]/div/div[3]/div/div/
button","desc":"Click '\u2715 button'", "input":null,
"field":"1180"}
141 # ✕ button
142 elem = page.get_by_role('button', name='✕', exact=True)
143 await elem.click(timeout=10000)
144
145 # -> Open the 'ALDA' table card to access session controls
(start session / generate QR).
146 # @@ts-step {"i":18,"type":"action","action":"click",
"selector":"xpath=/html/body/div[2]/div[2]/div/div[2]/div[2]",
"desc":"Click 'ALDA \u26aa Bo\u015f'", "input":null,
"field":"1137"}
147 # ALDA ● Boş
148 elem = page.get_by_text('ALDA ● Boş', exact=True)
149 await elem.click(timeout=10000)
150
151 # --> Assertions to verify final state
152
153 # --> Payment status did not show as completed – the ALDA

```

```

154         table modal displays "Bu masa şu anda boş".
155         # Assert-outcome: failed
156         # Assert: Expected payment status to show completed or paid.
157         await expect(page.locator("xpath=/html/body/div[2]/div[2]/div/
158         div[3]").nth(0)).to_contain_text("Bu masa \u015fu anda
159         bo\u015f", timeout=15000), "Expected payment status to show
160         completed or paid."
161
162         # --> The paid order or table session is not visible in the
163         customer session – all tables are shown as 'Boş' (empty).
164         # Assert-outcome: failed
165         # Assert: Expected the paid order or table session to remain
166         visible in the customer session.
167         await expect(page.locator("xpath=/html/body/div[2]/div[2]/div/
168         div[3]/div").nth(0)).to_contain_text("Bo\u015f",
169         timeout=15000), "Expected the paid order or table session to
170         remain visible in the customer session."
171
172         # --> Test blocked by environment/access constraints during
173         agent run
174         # Reason: TEST BLOCKED The test could not be run – an active
175         customer session could not be created through the UI, so the
176         payment status update cannot be observed. Observations: - The
177         Customer demo page shows an access error stating the QR is
178         invalid (visible text: 'QR kod artık geçerli değil') and retry
179         attempts did not recover the session. - The waiter (Garson)
180         ALDA table modal opens but only displays ...
181         raise AssertionError("Test blocked during agent run: " + "TEST
182         BLOCKED The test could not be run \u2014 an active customer
183         session could not be created through the UI, so the payment
184         status update cannot be observed. Observations: - The Customer
185         demo page shows an access error stating the QR is invalid
186         (visible text: 'QR kod art\u0131k ge\u00e7erli de\u011fil')
187         and retry attempts did not recover the session. - The waiter
188         (Garson) ALDA table modal opens but only displays ..." + " –
189         the exported script cannot reproduce a PASS in this
190         environment.")
191         await asyncio.sleep(5)
192
193     finally:
194         if context:
195             await context.close()
196         if browser:
197             await browser.close()
198         if pw:
199             await pw.stop()
200
201     asyncio.run(run_test())

```

Error

TEST BLOCKED The test could not be run — an active customer session could not be created through the UI, so the payment status update cannot be observed. Observations: - The Customer demo page shows an access error stating the QR is invalid (visible text: 'QR kod artık geçerli değil') and retry attempts did not recover the session. - The waiter (Garson) ALDA table modal opens but only displays 'Bu masa şu anda boş' and provides no visible control to start a session or generate a QR. - All tables in the waiter view are shown as empty ('Boş'), so there is no active table session with a pending payment to inspect.

Cause

The hosting side likely has no live table session available because QR/session state is being lost or never created in production. The customer QR is being treated as expired/invalid, while the waiter view shows every table as empty and offers no action to start a session, suggesting either a backend/session persistence bug, a broken session-creation endpoint, or an environment/configuration issue on Render that prevents active orders from being initialized and observed.

Fix

Make sure the production backend preserves table/session state and exposes the waiter flow needed to create an active customer session. Check for expired/misconfigured QR token generation, broken API endpoints for starting a table order/session, missing database persistence on Render, or environment variables causing the app to fall back to an invalid/empty session state. Verify that the waiter UI can create/open a table session and generate a fresh QR, and that the customer page resolves that QR to an active session instead of "QR kod artık geçerli değil". If the app relies on in-memory state, move session/table/payment state to a persistent store and ensure the production deploy is connected to the correct backend and database.

